Revision Summary and Detailed Response for Paper 201

MemForest: An Efficient Agent Memory System with Hierarchical Temporal Indexing

Dear Reviewers, Meta-Reviewer, and Associate Editor,

Thank you for the detailed comments. We revised the paper while preserving its architecture and central objective: reducing the freshness cost of persistent agent memory without sacrificing answer quality. Blue text marks substantive changes. All reviewer comments below are quoted verbatim. Due to the paper-length limit, detailed diagnostics, configurations, and supplementary tables are provided in the public complete version at reproducibility/paper/MemForest_full_revision.pdf. Code and result artifacts are hosted at https://github.com/Concyclics/MemForest; every repository reference below prints its clickable path explicitly.

REVISION SUMMARY AND RESPONSE TO THE META-REVIEW

This section also serves as the revision summary. Detailed evidence is not repeated here; each item refers to the corresponding reviewer response.

C-M1. *Strengthen the positioning and motivation of MemTree. Justify the need for a hierarchical temporal memory structure and compare it against simpler temporal alternatives (e.g., timestamped logs, validity intervals, temporal graphs or semantic timelines).*

Response. We added both a related-work comparison and a controlled experiment. The comparison identifies where timestamped event logs, tiered recent-memory designs, and bi-temporal graphs are used. The experiment compares Flat Log, Log-Time, Validity Interval, Scratchpad+Background, Triplet Graph, and MemTree under shared facts and evidence budgets. We also add Zep Local as a native temporal-graph system. Please refer to R3-W1, R3-W4, and R4-D3.

C-M2. *Clarify the write-path cost model and complexity analysis. Provide a detailed breakdown of the memory construction and maintenance pipeline, justify the assumptions underlying the complexity analysis (e.g., balanced-tree), and identify the dominant cost components.*

Response. We answer write cost, complexity, and the latency–token trade-off separately. The revised paper measures every write stage and limits the height-bounded statement to structural insertion and dirty-path dependency depth. End-to-end construction is not claimed to be logarithmic. Please refer to R1-W1, R3-W2, and R4-D1.

C-M3. *Provide a stronger and fairer empirical evaluation. Include additional temporal-memory baselines (such as Zep and simpler temporal alternatives), clarify timestamp handling and baseline configurations and reconcile discrepancies with previously reported results. The evaluation should characterize the quality/efficiency trade-off by jointly considering answer quality, write latency, query latency, token consumption, and LLM-call costs. Scalability should also be characterized (as memory size and the number of MemTrees grow).*

Response. We correct timestamp and retrieval-budget errors, add Zep Local, add equal-budget temporal controls, and evaluate three generative backbones on both benchmarks. The paper reports answer quality, isolated write rate, query latency, and a measured write-stage breakdown; request/token and scaling details are in reproducibility/paper/MemForest_full_revision.pdf. Please refer to R1-W1–W2, R3-W1–W2, R3-W5–W6, and R4-D14–D16.

C-M4. *Validate routing, retrieval, and temporal-memory robustness. Quantify the impact of routing decisions, planner behavior, evidence fragmentation, incorrect scope assignments and retrieval errors on final answer quality. The revision should also clarify how scope conflicts are handled.*

Response. We add reviewed routing and fragmentation diagnostics, planner controls, and scene-threshold sensitivity. Facts may be assigned to multiple scopes; session and scene paths remain available when an entity overlay is not activated. Please refer to R3-W3 and R4-D2, R4-D4, and R4-D5.

C-M5. *Clarify system semantics and implementation details. Provide detail regarding canonicalization, conflict resolution, retrieval scoring, tree traversal, merge policies and the handling of extraction errors. The revision should also specify freshness and consistency semantics.*

Response. The revised implementation defines canonicalization, multi-scope routing, retrieval, maintenance, merge, tree-level concurrency, and snapshot publication. Please refer to R3-W3 and R4-D8–D13.

C-M6. *Discuss the generality of the results beyond Qwen models.*

Response. We add a complete Gemma-4-12B-IT evaluation in which Gemma replaces every generative stage. Please refer to R4-D14.

C-M7. *Ensure full reproducibility. Make the experimental artifacts available and ensure that the evaluation complies with the conference reproducibility requirements (you can also find a precise list from one of the reviewers).*

Response. The single-blind revision links public code at https://github.com/Concyclics/MemForest, pinned baseline adapters, configurations, prompts, per-question outputs, logs, and offline validators. The public complete-version PDF contains material omitted from the length-limited paper. The artifact index is reproducibility/README.md. Please refer to R1-W3 and R3-W5–W6.

DETAILED RESPONSE TO REVIEWER #1

R1-W1. *It would be better if Section 5.2 explains the latency-token tradeoff more, such as how to control the tradeoff from a practical perspective.*

Response. Yes. We now answer the existence, cause, and control of this trade-off in Sections 4.1 and 6.1, supported by Figure 7.

Existence and cause. Revised Section 4.1 states:

> *This creates a latency–token trade-off: smaller chunks expose more parallel requests and bound each generation, but repeat prompt/schema input and JSON scaffolding; larger chunks amortize that overhead but reduce parallelism and can lose answer-bearing evidence as context grows. For short sessions or below serving saturation, repeated prefill can outweigh the parallel-latency benefit.*

Control. Revised Section 6.1 states:

> *Figure 7 reports a controlled assembled-long-session sweep using Gold Coverage (answer-bearing gold-span retention), facts/s, tokens/fact, extracted-fact count, and total input-plus-output tokens. Larger chunks lower total token work chiefly by emitting fewer facts, which also lowers extraction fidelity. Gold Coverage falls and tokens/fact ultimately worsens as answer-bearing evidence is omitted, consistent with the documented tendency of long-context LLMs to underuse evidence in the middle of their inputs [18]. A practical rule is to choose the largest local window that preserves extraction fidelity while still exposing enough independent cells to saturate the serving backend. We select $b = 2$ because it preserves 100% Gold Coverage, remains near the highest throughput, and uses fewer tokens per fact than $b = 1$.*

R1-W2. *I wonder why LoCoMo is the harder benchmark for MemForest than LongMemEval. Section 2.3.3 says that LoCoMo has more temporal-reasoning questions than LongMemEval-S, and because MemForest focuses on temporal management, I expected that MemForest would have the best pass@1 accuracy also in LoCoMo. The authors mention that LoCoMo has more entangled evidence than LongMemEval, but more explanations would be helpful.*

Response. We corrected the evaluation protocol and the LoCoMo category description.

Aligned re-evaluation. Revised Section 5.1 states:

> *We rejudge every frozen main-table answer with* `deepseek-v4-flash` *at temperature zero and the corresponding released public benchmark prompt. The LoCoMo headline follows the pinned Mem0 and EverMemOS runners and covers categories 1–4 (1,540 questions); category 5 is reported separately in the public complete version.*

The rejudged results are reported in Table 4, with category 5 in reproducibility/paper/MemForest_full_revision.pdf. The released LoCoMo public prompt applies more permissive acceptance criteria than our strict diagnostic prompt; please refer to R3-W6 for the paired sensitivity analysis.

Category-mapping correction. Revised Section 2.2.2 states:

> *In LoCoMo, temporal questions are raw category 2, with 321/1,986 questions (16.2%); the 841-question block is the single-hop category.*

We apologize for the labeling error in the submitted paper.

R1-W3. *It would be better if source codes and experimental setups were available.*

Response. We apologize for this omission. The repository URL was present in the submission system but was omitted from the submitted manuscript. The revised first page now states:

> *PVLDB Artifact Availability: The source code, data, and/or other artifacts have been made available at https://github.com/Concyclics/MemForest.*

Revised Section 5.1 further states:

> *Pinned configuration and validation records are available in the public baseline guide and complete version.*

The repository paths are reproducibility/BASELINES.md and reproducibility/manifests/baseline_versions.json; the complete PDF is reproducibility/paper/MemForest_full_revision.pdf.

DETAILED RESPONSE TO REVIEWER #3

R3-W1. *Target scenarios need sharper justification. The temporal motivation is strong, but MemTree itself is not fully justified. Simpler designs such as timestamped logs, validity intervals, temporal graphs, or semantic timelines might also handle the example queries. The paper should compare with stronger temporal baselines, at least a timestamp-aware log and a temporal graph/timeline method.*

Response. Revised Sections 7 and 6.2 separate native end-to-end systems from shared-fact representation controls, summarized below.

Reviewer design	Native end-to-end baseline	Shared-fact control
Timestamped log / timeline	MemPalace; EverMemOS	Flat Log; Log-Time
Recent scratchpad + background index	LightMem; MemoryOS	Scratchpad+Background
Validity intervals	Zep/Graphiti bi-temporal validity	Validity Interval
Temporal graph	Zep Local / Graphiti	Triplet Graph

Tables 3 and 4 report the native systems, including Zep Local. Table 8 then reports the controlled comparison described in the paper:

> *Flat Log, tuned Log-Time, Validity Interval, Scratchpad+Background, Triplet Graph, and MemTree use the same canonical facts, timestamps, embeddings, and final fact budgets. Tuned Log-Time leads by 0.7 points at $k = 10$, and the two are effectively tied at $k = 30$. MemTree has the highest observed coverage at $k = 50$ (92.0%), supporting its main role as a structured high-recall candidate source when more evidence can be retained.*

R3-W2. *Efficiency needs a clearer breakdown. The paper risks making updates sound logarithmic, but the write path still includes LLM extraction, routing, embedding, refresh, and index maintenance. These costs should be separated more clearly, including when LLM work or dirty-node refresh dominates. The LoCoMo results also need a clearer quality–efficiency comparison, especially against EverMemOS.*

Response. Revised Section 5.4 and Figure 8 state:

> *The two purely autoregressive stages, extraction and dirty refresh, account for 83.58% of measured build time. Adding hybrid canonicalization raises the LLM-involved share to 90.57%. Structural insertion, index update, and persistence together remain below 1%. An insertion marks the summaries of affected parents and ancestors dirty: a parent depends on its refreshed children, but nodes at the same level or in different trees do not depend on one another. Using the exact stage tokens and historical single-request throughput under the matched Qwen/vLLM configuration, the throughput-calibrated serial replay takes 54.58 minutes for extraction and 9.88 minutes for dirty refresh. The measured parallel stages take 122.31 and 27.92 seconds, yielding 26.8× and 21.2× speedups, respectively.*

The public complete version (reproducibility/paper/MemForest_full_revision.pdf) reports the replay model, sensitivity range, calls, token/request diagnostics, and method-by-method released-path dependencies. Its released trace inputs are at reproducibility/results/write_path_traces/.

Revised Section 4.4 states:

> *End-to-end construction is not logarithmic. Under bounded fan-out and balanced MemTrees, only structural insertion and the dependency depth of a dirty path are height-bounded; extraction still scales with incoming content, and total refresh work scales with the distinct dirty nodes.*

Figure 1 provides the requested LongMemEval and LoCoMo quality–efficiency comparison, including EverMemOS and Zep Local.

R3-W3. *Retrieval robustness and freshness are unclear. MemForest relies on routing facts into the right trees, but the paper does not measure routing or scene-clustering errors. The lazy-refresh design also needs clearer semantics: what becomes visible after a write, whether summaries may be stale, and how concurrent reads and writes are handled.*

Response. We address this comment in three aspects.

(1) Routing accuracy and scene stability. Revised Section 6.2 reports the reviewed routing, fragmentation, and scene-threshold results; the complete-version routing table in reproducibility/paper/MemForest_full_revision.pdf provides the underlying measurements and labels. The revised paper concludes:

> *These results support a high-precision but deliberately redundant routing design: active entity routes are usually valid, but no single overlay is complete and fragmentation remains material on LongMemEval. MemForest therefore retains multi-scope placement, fact-to-tree union recall, and multi-tree browse instead of committing each fact to one route.*

Reviewed routing labels and fragmentation summaries are available at reproducibility/results/semantic_audit/author_adjudicated/.

(2) Lazy refresh. Revised Section 4.3 states:

> *Here "lazy" means deferral within a write batch: records are grouped by touched tree, dirty trees and ancestors are deduplicated, and each touched tree is refreshed once before the write completes.*

Table 2 reports cross-session overlap with and without the global `entity:user` fallback; Section 5.6 provides the subforest migration/merge path used to shard that fallback. The released audit path is reproducibility/results/write_conflicts/.

(3) Concurrency and visibility. Revised Section 4.5 states:

> *In the online asynchronous implementation, each MemTree owns an independent reader–writer lock. Insertion and summary refresh take the write lock of only the touched tree; operations on distinct trees are scheduled concurrently, while same-tree writes serialize. Range queries, browse primitives, and snapshot export use shared read locks, so concurrent reads proceed without conflicting with one another and wait only for a writer on the same tree. Snapshot save renames the completed temporary directory into place.*

The implementation directory is reproducibility/implementation/async_core/. The relevant clickable locations are `bplustree.py`, lines 21–54 for the reader–writer lock, `forest.py`, lines 1425–1476 for cross-tree insertion and dirty-set snapshotting, and `forest.py`, lines 2036–2106 for atomic snapshot publication.

R3-W4. *W4: §2.3 overstates the limitations of prior work. §2.3 claims existing systems cannot answer historical-state ("what was true before X") queries. This holds for an idealized pure-vector or pure-profile design, but not for the temporally-structured systems the paper also cites and benchmarks. Zep [1] solves it by construction—a bi-temporal graph where superseded edges are invalidated but kept, with a non-lossy episodic store. Yet it is cited and excluded from the experiments: the most temporally capable baseline is omitted while the paper claims existing systems can't do temporal. The authors' own traces [2] refute the premise. The released EverMemOS answers retrieve the correct, correctly-time-stamped evidence on essentially every temporal question (e.g., "[April 6, 2023] attended the Maundy Thursday service"). The historical state is present and retrieved—not lost.*

Response. We address this comment in three parts.

(1) Scope of the temporal claim. Revised Section 2.2.2 states:

> *Many widely used flat-fact, profile, and tiered designs do not expose validity-aware temporal navigation as a first-class index. Other designs provide explicit temporal organization: timestamped logs, validity intervals, temporal graphs, semantic timelines, and MemTrees can preserve historical evidence;*

Zep/Graphiti stores bi-temporal graph edges with retained provenance.

(2) Zep repository and added Zep Local baseline. At submission time, the official https://github.com/getzep/zep repository exposed examples, integrations, and tools that call Zep Cloud, not the complete Zep product or service implementation. It therefore did not provide a version-pinned, self-hosted implementation for a controlled local run. Following the reproduction procedure of Zhou et al. [42], we subsequently use the open temporal-graph implementation in https://github.com/getzep/graphiti to add Zep Local. Revised Section 5.1 states:

Following Zhou et al., we reproduce Zep Local from the open Graphiti temporal-graph core, Neo4j, and self-hosted generation and embedding services. This is a local Graphiti realization rather than Zep Cloud.

Tables 3 and 4 report the added end-to-end results; the complete version (reproducibility/paper/MemForest_full_revision.pdf) specifies the pinned Graphiti/Neo4j recipe and native retrieval objects. Our released runner is reproducibility/baselines/zep_local/run_benchmark.py: lines 62–67 pin Graphiti, embedding, and retrieval versions; lines 395–423 configure Graphiti/Neo4j and self-hosted services; and lines 603–610 and lines 724–740 implement native episode ingestion and retrieval.

(3) EverMemOS. Revised Section 7 states:

EverMemOS preserves event timestamps in its memory units and event logs. Its agentic search combines hybrid retrieval with an LLM sufficiency check and, when needed, a second serial query-refinement round. It can therefore recover historical evidence through query-time LLM reasoning rather than requiring the index itself to expose an explicit temporal-navigation path.

R3-W5. *W5: The temporal-category gap might be a timestamp-configuration artifact, not the substrate. Mem0: the conversation timestamps seem not to be stored with the event. The dates in Mem0's Temporal benchmark outputs seem to be 2026, instead of the event's actual date 2023. If the event is not carried with a timestamp, every "how many days ago" question is unanswerable. This is configuration, not capability: Mem0's* `timestamp` *parameter has existed at-least since late-2025. [3] Scale: A lot of LongMemEval temporal questions are date arithmetic ("how many days/weeks ago"), so these issues drive the category score. [2] Conclusion: the comparison conflates substrate quality with per-system timestamp/reference-date handling, so the temporal win cannot be attributed to MemTree.*

Response. We thank the reviewer for identifying this issue. We audited timestamp propagation in all evaluated adapters and found this specific benchmark-to-runtime timestamp mismatch only in our submitted Mem0 adapter. We corrected the adapter to write source event time to both the API timestamp argument and metadata, then reran every affected Mem0 cell with session-bounded LoCoMo writes, complete message-coverage checks, runtime-date rejection, and a global top-10 flat retrieval budget. The refreshed results replace the submitted Mem0 values in Tables 3 and 4. The timestamp write is implemented at reproducibility/baselines/mem0/mem0_adapter.py (lines 286–331); rerun outputs and validation records are at reproducibility/results/mem0_corrected/.

R3-W6. *W6: Baseline temporal scores fall far below the systems' own published numbers, and only on temporal. The baselines here underperform their own reported results almost entirely on the temporal category—an external check that corroborates W2: EverMemOS: its own paper ([4] Table 2, GPT-4.1-mini) reports 77.4% temporal and 89.7% knowledge-update; here (Qwen3-30B) it scores 52.5% temporal but 86.7% knowledge-update—knowledge update essentially unchanged (-3), temporal collapses (-25). Mem0: In EverMemOS's paper, 72.18% temporal in that table vs 27.8% here, its temporal drop (∼-44) roughly doubles its knowledge-update drop. One might attribute the temporal drop to the weaker backbone (Qwen3-30B vs GPT-4.1-mini). But W2 shows the failures are missing dates, not failed arithmetic—so the backbone is not the cause, and the shortfall is more-likely the configuration, not the baselines' temporal capability. References: [1] https://arxiv.org/abs/2501.13956; [2] https://github.com/Concyclics/MemForest/tree/main; [3] https://github.com/mem0ai/mem0/issues/3256; [4] https://arxiv.org/abs/2601.02163.*

Response. We reconcile the public-score gap across six protocol variables: timestamp policy, retrieval budget, pipeline version, answer interface, answer backbone, and judge prompt. We make two changes.

First, one fixed DeepSeek-V4-Flash judge evaluates every maintable answer with the corresponding released public prompt. Second, the complete version (reproducibility/paper/MemForest_full_revision.pdf) reports a fixed-answer strict/public sensitivity test. On LoCoMo temporal, the public prompt changes EverMemOS, MemForest, and Mem0 by +17.33, +16.00, and +5.33 points, respectively. Thus judge policy materially moves all systems, but it does not by itself explain every public-score gap. The released sensitivity summary is reproducibility/results/judge_prompt_sensitivity/summary.csv. Mem0 top-50/top-200 diagnostics are released separately at reproducibility/results/mem0_corrected/top50_top200_control/ rather than being mixed with the global top-10 main table.

DETAILED RESPONSE TO REVIEWER #4

R4-D1. *The O(log N) complexity claim for MemForest assumes a balanced tree, but the paper never proves or empirically verifies that the tree remains balanced under continuous real-world writes, which can be highly skewed toward certain entities or scopes.*

Response. Please refer to R3-W2 for the narrowed complexity statement. The complete-version audit covers 250,118 trees from two full Qwen builds, observes maximum height four, and finds no violation of the implementation's bounded-fan-out height check. This is empirical implementation evidence, not a proof for arbitrary write sequences.

R4-D2. *The "temporal scope" abstraction is central to the paper but is never formally evaluated in isolation—there is no experiment showing that scope assignment is accurate or consistent.*

Response. Please refer to R3-W3. We add reviewed entity-routing, fragmentation, scene-stability, and tree-family diagnostics to the complete version; R3-W3 prints the released data path.

R4-D3. *The paper does not consider whether a simple unified index that combines a scratch pad for the k-recent sessions plus a time-sensitive index that is constructed in the background (using techniques similar to dreaming from Claude) might be sufficient to address write overheads and ensure that recent sessions are immediately querable.*

Response. Added. Scratchpad+Background is one of the shared-fact controls in Section 6.2. Please refer to R3-W1 for the experiment and its scope.

R4-D4. *The three tree families are described as complementary, but the paper never explains how scope conflicts are resolved when the same fact is relevant to multiple entities or scenes, and the routing logic discussion is too brief to evaluate this.*

Response. Revised Section 4.1 states:

> *A fact may enter multiple scopes; multi-assignment is not resolved by forcing one winning tree.*

Section 6.2 and the complete version report the routing and tree-family diagnostics; please also refer to R3-W3.

R4-D5. *Scene trees rely on clustering to group semantically coherent situations, but the actual method is never described in sufficient detail to evaluate whether scene definitions are stable or meaningful.*

Response. Revised Section 4.1 specifies scene assignment and merge thresholds. Section 6.2 and the complete-version routing table report threshold stability and motivate redundant multi-scope placement. Please refer to R3-W3.

R4-D6. *The chunk size parameter is inadequately motivated— beyond the reported sweep in Appendix C, the paper offers no rule of thumb for practitioners, nor does it discuss whether different agentic tasks or types of interactions/sessions require different batching strategies.*

Response. Please refer to R1-W1 and revised Section 6.1, which report the measured sweep and practical selection rule.

R4-D7. *The cost argument for parallel extraction over a single LLM pass lacks nuance. For shorter sessions where the context fits comfortably in one pass, there is likely a tradeoff with prefill complexity that is never analyzed.*

Response. Please refer to R1-W1 and revised Section 4.1, which state the short-session and serving-saturation boundary.

R4-D8. *Canonicalization is described at a high level (normalizing surface forms, merging duplicates) but the paper provides no detail on how it is implemented—whether LLM-based, rule-based, or embedding-based—nor what prompts or consistency guarantees are used. Critically, when two parallel chunks produce contradictory facts, it is unclear how the system resolves the conflict.*

Response. Revised Section 4.1 states:

> *Canonicalization uses normalized exact matching, embedding-retrieved candidates, and a bounded LLM equivalence check. Equivalent records merge provenance and occurrence times; contradictory or otherwise non-equivalent updates remain separate facts with time anchors.*

The public implementation is src/extraction/dedup.py (lines 85–129) for the bounded LLM equivalence check and lines 158–220 for normalized/embedding candidate filtering. Its prompt is src/prompt/dedup_prompt.py (lines 11–34). Evaluated thresholds are recorded in reproducibility/manifests/runtime_configs.json (lines 11–23).

R4-D9. *The paper never discusses what happens when LLM extraction produces hallucinated or factually incorrect facts—there is no noise or error propagation analysis.*

Response. Revised Sections 4.1 and 4.5 state:

> *Canonicalization consolidates records but does not verify their truth. Extraction errors may propagate into multiple scopes and summaries; provenance supports targeted deletion and refresh.*

R4-D10. *The similarity or distance function used during retrieval is never specified (cosine, dot product, L2), nor is its accuracy analyzed.*

Response. Revised Section 4.2 specifies cosine similarity over normalized Qwen3 embeddings. The equal-budget coverage control in Section 6.2 reports the resulting evidence access. The exact normalized inner-product implementation (equivalent to cosine) is at src/build/node_index.py (lines 113–121) and lines 182–202.

R4-D11. *The retrieval implementation is underspecified—it is unclear whether FAISS or another ANN library is used, what index structure backs it, and how the tree union is computed in practice.*

Response. Revised Section 4.2 specifies the persisted RootIndex and NodeIndex, top-k root recall, candidate union and deduplication, and branch descent. The evaluated per-user indexes use exact cosine search rather than an ANN approximation. Tree recall and candidate union are at src/query/retriever.py (lines 69–108); index persistence and exact search are at src/build/node_index.py (lines 137–202).

R4-D12. *The term "promising child node" is used in the tree browse description without ever being formally defined. In embedding-only mode it presumably means highest similarity, but in LLM-guided mode the selection criterion is never stated.*

Response. Revised Section 4.2 defines it explicitly. Embed selects the highest cosine-scoring children. Planner-guided browsing uses a generated tree-specific subquery and an LLM branch-selection prompt over child summaries. The two branch-selection paths are at src/query/browser.py (lines 95–162) and lines 218–262.

R4-D13. *The decision criteria for merge operations are never explained—the paper does not specify what triggers a merge, what the decision logic is, or what the downstream impact on retrieval recall is.*

Response. Revised Section 4.3 separates canonical-fact merge from tree rebalance and memory migration. Canonical merge requires equivalence; tree rebalance follows the bounded-fan-out invariant; migration combines canonical facts and scope assignments before regenerating affected derived summaries and indexes. The main migration experiment remains in Section 5.6. The corresponding source paths are src/extraction/dedup.py (lines 158–237), src/build/tree.py (lines 236–321), and src/forest/forest_merge.py (lines 246–420).

R4-D14. *The evaluation uses only two Qwen3 model variants. It is unclear whether results generalize to other LLM families (e.g., Llama, Mistral), making the improvements potentially model-family-specific.*

Response. We add Gemma-4-12B-IT to both complete benchmarks and replace every generative stage, while keeping the embedding model fixed as a control. The mixed rankings in Tables 3 and 4 show cross-family applicability and model-dependent answer quality.

R4-D15. *Not all baselines discussed in related work are included consistently across all experiments—for example, MemPalace is absent from the migration experiment and the retrieval ablation despite being the closest design point.*

Response. MemPalace remains in every end-to-end answer-quality table. The migration experiment compares systems that expose a direct merge of already materialized memory states; MemPalace exposes continued raw-history writes but no comparable native merge. The revised retrieval ablation is representation controlled: the timestamped Flat Log is the shared-fact counterpart of a fidelity-first history, while native MemPalace remains in the main tables.

R4-D16. *There is no latency analysis under increasing memory size. The paper doesquery-time latency scales as the number of MemTrees grows, which is critical for long-horizon deployments.*

Response. The complete version composes 1, 2, 4, and 8 frozen forests, increasing state from 1,428 facts and 250 trees to 10,781 facts and 1,868 trees. Under fixed top-10 Embed retrieval, p95 remains 41.79–44.13 ms. This microbenchmark excludes Planner, answer generation, construction, and loading; those limits are stated with the result.

MemForest: An Efficient Agent Memory System with Hierarchical Temporal Indexing

Han Chen
National University of Singapore
Singapore, Singapore
chenhan@u.nus.edu

Zining Zhang
National University of Singapore
Singapore, Singapore
zzn@nus.edu.sg

Wenqi Pei
National University of Singapore
Singapore, Singapore
wenqi_pei@u.nus.edu

Bingsheng He
National University of Singapore
Singapore, Singapore
dcsheb@nus.edu.sg

Ming Wu
Zero Gravity Labs
United States
ming@0g.ai

Jason Zeng
Zero Gravity Labs
United States
jason@0g.ai

Michael Heinrich
Zero Gravity Labs
United States
michael@0g.ai

Wei Wu
Zero Gravity Labs
United States
wei@0g.ai

Hongbao Zhang
Zero Gravity Labs
United States
peter@0g.ai

ABSTRACT

Memory is a fundamental component for enabling long-context LLM agents, supporting persistent state across interactions through a continuous serve-and-update lifecycle. Despite substantial prior work, existing systems suffer from significant maintenance overhead due to two key limitations: coarse-grained state management and sequential dependencies in their released update pipelines. In particular, updates are often tightly coupled with LLM inference and require full-state rewrites, leading to poor scalability and growing latency as memory accumulates. To address these challenges, we present **MemForest**, a memory framework that reformulates agent memory as a write-efficient temporal data management problem. MemForest breaks the sequential bottleneck via parallel chunk extraction, decoupling memory construction into concurrent, independent operations. To further eliminate coarse-grained maintenance, we introduce *MemTree*, a hierarchical temporal index that organizes memory as time-ordered trees rather than flat global summaries. This design replaces full-state rewrites with localized per-node updates, reducing maintenance cost to the affected tree paths while preserving temporally evolving states. We evaluate MemForest on LongMemEval-S and LoCoMo. Under the aligned public evaluation protocol, MemForest reaches 81.8% pass@1 on LongMemEval-S and 84.09% on LoCoMo with Qwen3-30B, while sustaining isolated write rates 6.0× and 9.5× those of EverMemOS. A complete Gemma-4-12B evaluation and a reproduced Zep Local baseline further test model-family and temporal-representation generality.

KEYWORDS

LLM agents, agent memory, persistent memory, temporal indexing, hierarchical retrieval, write-efficient memory

PVLDB Reference Format:
Han Chen, Zining Zhang, Wenqi Pei, Bingsheng He, Ming Wu, Jason Zeng, Michael Heinrich, Wei Wu, and Hongbao Zhang. MemForest: An Efficient Agent Memory System with Hierarchical Temporal Indexing. PVLDB, 20(1): XXX-XXX, 2027.

doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Concyclics/MemForest.

1 INTRODUCTION

Large language model (LLM) agents are increasingly expected to sustain personalized and stateful behavior across interactions that span days, weeks, or months [24, 26, 41]. This requirement arises in applications such as conversational assistants, long-lived task agents, and interactive social agents, where useful behavior depends on preserving user preferences, prior commitments, and accumulated experiences over time. This, in turn, requires an efficient and effective memory system that transforms interaction streams into a structured memory state that remains useful as evidence accumulates and user state evolves. Recent memory systems have made substantial progress in managing long-context interactions through hierarchical structures, online/offline consolidation, and temporal graphs [3, 9, 13, 17, 27]. At the same time, recent database systems work has begun to treat LLM+retrieval workloads as first-class data systems, optimizing retrieval–inference pipelining, cache reuse, and persistent vector infrastructures for LLM applications [1, 14, 31, 37]. In this paper, we focus on improving the efficiency and effectiveness of persistent agent memory systems under this systems perspective.

Current memory systems can be viewed as having three core functions: *extraction*, which converts raw interactions into persistent memory records; *retrieval*, which fetches relevant context for downstream response generation; and *maintenance*, which updates, consolidates, and restructures existing knowledge over time [9, 40]. While prior work has substantially improved retrieval quality, storage organization, and query-time reasoning, their online write paths still commonly rely on synchronous extraction, consolidation, or

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

profile-style maintenance over existing memory state [3, 9, 13, 17]. In realistic deployments, the dominant bottleneck shifts to these write-heavy paths, driven by synchronous LLM inference overhead and repeated full-state updates. As illustrated in Figure 1, the dominant latency in representative systems stems from *extraction* and *maintenance*, rather than *retrieval*.

We identify two structural bottlenecks behind this inefficiency. First, existing architectures often embed the LLM directly within the write critical path of *extraction* and *maintenance*. In the evaluated released pipelines, state-dependent extraction, summarization, reconciliation, or profile maintenance introduces serial dependencies before new evidence becomes queryable. This creates a severe latency bottleneck that worsens as interaction frequency increases. For example, systems such as EverMemOS rely on write-time semantic processing and consolidation, which improves memory quality but also places substantial LLM work directly on the update path [13]. Second, current systems often operate at a coarse granularity, routinely requiring the model to perform full-state rewrites of compact hot states such as user profiles or global summaries [3, 13, 17, 24]. Even when only minor new evidence arrives, the system must reread and rewrite the entire memory object. As memory accumulates, this imposes a maintenance cost and latency floor that scales with maintained-state size rather than with newly arrived evidence.

However, improving write efficiency alone is not sufficient, because long-context agent memory is inherently temporal [11, 20, 27, 33]. User states evolve, facts are revised, and older information often remains necessary for complex reasoning. For example, if a user first lived in Boston, later moved to New York, and then relocated to San Francisco, a memory system should support not only the current-state query of where the user lives now, but also historical and transition queries such as where the user lived before New York and when the move occurred. We therefore frame long-context agent memory as a write-efficient temporal data management problem, in which persistent memory must remain incrementally maintainable while preserving historical state evolution [2, 8]. The core challenge is to overcome the trade-off between write efficiency and faithful temporal memory representation: a system must minimize the serial delays of *extraction* and the state-size-dependent costs of *maintenance* in order to maximize update throughput, while rigorously preserving historical states for long-context reasoning in order to maximize answer accuracy.

In this paper, we propose **MemForest**, a memory architecture designed around this write-efficient temporal objective. MemForest combines parallel chunk extraction, canonical fact consolidation, and *MemTree*—a hierarchical temporal index that materializes scoped memory as time-ordered trees rather than flat records or repeatedly rewritten profiles. MemForest adopts a similar high-level intuition to write-optimized indexing in database systems, where update costs are reduced by avoiding repeated rewrites of compact indexed state, as exemplified by LSM-trees [23], although the maintained object here is persistent agent memory rather than key-value state. Parallel chunk extraction dismantles the serial *extraction* bottleneck by decoupling and processing new interactions concurrently. Canonical fact consolidation repairs the semantic fragmentation inherently introduced by such parallelization. To optimize *maintenance*, MemTree replaces full-state rewrites with

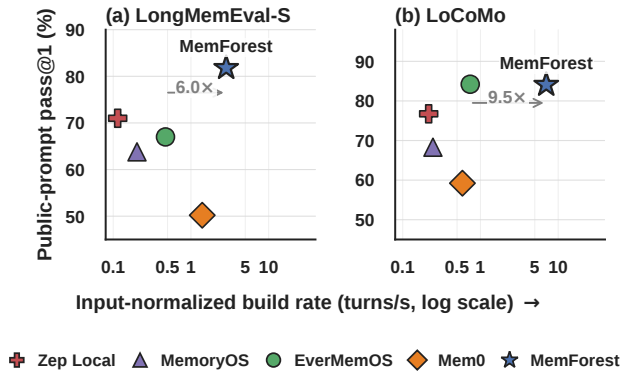

Figure 1: Qwen3-30B build-rate–accuracy operating points. Accuracy uses each full benchmark split. Build rate uses isolated native writes for one LongMemEval instance and one matched LoCoMo conversation; Section 5.4 reports the measurement scope.

localized per-node updates and lazy summary regeneration, so that index-maintenance cost scales with the affected tree paths and distinct dirty nodes rather than with the total accumulated memory size (Section 4.4). At query time, MemForest leverages this structure to perform coarse-to-fine *retrieval*, navigating from broad interval summaries down to precise leaf-level evidence [7, 29, 30].

We evaluate MemForest on two long-context memory benchmarks, LongMemEval-S and LoCoMo [20, 33]. Under the aligned public protocol, MemForest reaches 81.8% pass@1 on LongMemEval-S and 84.09% on LoCoMo categories 1–4 with Qwen3-30B. Its isolated build rate is 6.0× and 9.5× that of EverMemOS on the corresponding measurements. A complete Gemma-4-12B setting and a reproduced Zep Local baseline test model-family and temporal-graph generality; the Gemma results show that answer-quality rankings remain model-dependent. This efficiency matters because, in long-horizon deployments, *extraction* and *maintenance* costs are paid repeatedly as new sessions arrive rather than once as offline preprocessing. Overall, MemForest improves the memory substrate by accelerating write operations, explicitly preserving temporal state evolution, and exposing retrieval across multiple granularities.

Our contributions are threefold:

- We identify serial LLM-in-the-loop extraction and the state-size-dependent latency of full-state maintenance rewrites as the two dominant structural limitations of long-context agent memory systems.
- We introduce MemForest, a memory architecture that resolves these bottlenecks by combining parallel extraction with hierarchical temporal indexing, enabling localized updates, variable-granularity retrieval, and a persistent, queryable, and temporally evolving memory substrate under continuous writes.
- We show that this architectural shift improves the speed–accuracy trade-off on LongMemEval-S, where MemForest is the strongest overall system among the evaluated stateful baselines, while remaining competitive on LoCoMo with

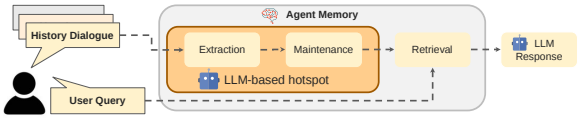

Figure 2: A generic workflow of agent memory systems. New dialogue history first goes through *extraction*, then *maintenance* updates the existing memory state, and future user queries trigger *retrieval* over the maintained memory. In many existing systems, the dominant LLM hotspot lies on the write path of extraction and maintenance.

substantially reduced write-path cost across both benchmarks.

The remainder of this paper is organized as follows. Section 2 introduces the workload model and problem formulation for long-context agent memory. Section 3 presents the system overview of MemForest and its core structure, MemTree. Section 4 provides the design and implementation details of its extraction, retrieval, and maintenance workflows. Section 5 evaluates MemForest on LongMemEval-S and LoCoMo. Section 6 provides ablation studies and analysis of the key design choices. We review related work in Section 7, and conclude this paper in Section 8.

2 BACKGROUND AND PROBLEM FORMULATION

2.1 Workload Model

We consider a long-lived agent that interacts with a user through a sequence of sessions

$$\mathcal{D} = (S_1, S_2, \ldots, S_T), \tag{1}$$

where each session S_t is a multi-turn interaction arriving over time. The system must continuously extract new sessions, maintain memory across the full interaction horizon, and answer future queries against the accumulated memory state. This workload has three key properties. First, new sessions should become queryable without requiring full reconstruction of prior memory. Second, once written, memory may be queried many times by downstream tasks, including current-state lookup, multi-session recall, knowledge updates, and temporal reasoning. Third, practical systems may need to support merge, deletion, or migration as memory policies evolve over time [13, 19, 27, 32]. This setting is reflected in long-context benchmarks such as LoCoMo and LongMemEval [20, 33].

Existing memory systems typically consist of three stages: *extraction*, which converts newly arrived interactions into memory records; *maintenance*, which updates, merges, reorganizes, or refreshes existing memory state; and *retrieval*, which recalls relevant memory for downstream response generation [9, 17, 40]. Figure 2 illustrates this common workflow. Unlike one-shot long-context prompting, memory construction is therefore not a one-time preprocessing step, but part of an ongoing mixed read-write workload.

2.2 Deficiencies of Existing Systems

As discussed in the Introduction, we identify two major limitations in existing systems: a shared write-path bottleneck that constrains incremental memory construction, and the fact that this bottleneck becomes even more problematic in temporal memory settings.

2.2.1 Bottlenecks of Existing Systems. Representative systems use different temporal representations. Across the released pipelines evaluated in this paper [3, 9, 13, 17, 22, 27], LLM extraction or state-dependent maintenance often remains on the write critical path. Table 1 separates representation from this implementation-level write dependency.

This dependency appears in multiple forms. Some systems maintain compact per-user objects, such as profiles, summaries, or core-memory documents, and update them through read–modify–write maintenance [3, 13, 17, 24]. Others preserve broader event, index, or graph state and pay extraction, linking, or consolidation costs before an update becomes queryable [9, 22, 27]. These are different operating points between compact state, write-time processing, and query-time reconstruction.

As a result, the evaluated implementations can become expensive under writes in two ways. First, *extraction* uses synchronous LLM calls that extract, summarize, or reconcile memory before new evidence becomes queryable [9, 13, 17]. Second, *maintenance* can operate on coarse-grained state that is reread and rewritten when new evidence arrives [3, 13, 17]. Parallelism across users does not shorten this per-instance dependency path.

2.2.2 Issues in Temporal Memory Systems. This write-path bottleneck is especially problematic for temporal memory [11, 20, 27, 33]. Long-context memory must preserve not only what is true now, but also what used to be true and when transitions occurred. For example, if a user lived in Boston, later moved to New York, and then relocated to San Francisco, the system should support current-state, historical-state, and transition queries over the same trajectory.

Many widely used flat-fact, profile, and tiered designs do not expose validity-aware temporal navigation as a first-class index. Other designs provide explicit temporal organization: timestamped logs, validity intervals, temporal graphs, semantic timelines, and MemTrees can preserve historical evidence; Zep/Graphiti stores bi-temporal graph edges with retained provenance [27]. Latest-state summaries are convenient for current-state lookup, while broad logs may shift temporal reconstruction to query time and graphs add extraction and edge-maintenance work. The challenge is therefore not merely to store more memory, but to define a stable write unit, preserve evolving state, and refresh only affected access paths. In LoCoMo, temporal questions are raw category 2, with 321/1,986 questions (16.2%); the 841-question block is the single-hop category.

2.3 Problem Formulation

We first introduce the following concepts.

Persistent state consists of the canonical memory contents that define the durable semantics of the system. In MemForest, this includes canonical facts, fact-to-tree mappings, cluster states, session references, and the tree structures that preserve temporally organized memory within each scope.

Table 1: Representative temporal-memory accumulation designs and native write paths.

System	Persistent representation	Temporal accumulation	Released write path
Mem0	Indexed fact store	Timestamped facts under configured ingestion metadata	LLM ADD/UPDATE/DELETE decision over retrieved candidates
MemoryOS	Short-, mid-, and long-term tiers	Recent memory with promotion and background consolidation	Session-chain update and state-dependent promotion
EverMemOS	Event logs and consolidated memory	Timestamped events plus semantic consolidation	Episode/event extraction followed by consolidation
LightMem	Compressed indexed memories	Recent online records plus offline/background updates	Extraction/compression with state-dependent updates
MemPalace	Fidelity-first raw history	Timestamped raw events	Lightweight indexing; abstraction deferred to retrieval
Zep Local	Bi-temporal graph and episodic store	Versioned graph edges with retained provenance	Ordered episode extraction and graph maintenance
MemForest (ours)	Scoped, time-ordered MemTrees	Canonical facts plus interval summaries	Parallel extraction and local dirty-path refresh

Derived artifacts are auxiliary structures materialized from persistent state to accelerate access. In MemForest, these include interval summaries, node embeddings, and root-level index entries. They improve efficiency, but they are not the source of truth and can be selectively refreshed after writes.

Write-path latency measures the wall-clock time required to convert newly arrived interaction history into queryable memory. This is the primary systems metric for write responsiveness.

Token cost measures the total model input/output usage incurred during memory construction and maintenance. This captures model-side resource consumption even when latency is partially hidden by parallelism.

Given a continuous session stream $\mathcal{D}$, our goal is to maintain a persistent memory substrate $\mathcal{M}$ that improves the efficiency of continuous memory construction while preserving the fidelity of temporally evolving state. In particular, we seek a design whose write cost is governed by the size of the incoming update rather than the size of the entire accumulated memory, and whose systems behavior can be evaluated along two primary dimensions: write-path latency and token cost. MemForest addresses this objective through three design choices: it uses canonical facts as stable write units, separates persistent state from derived access artifacts, and materializes scoped memory as a temporal index that supports localized maintenance and coarse-to-fine retrieval.

3 MEMFOREST OVERVIEW

MemForest is a persistent temporal memory substrate for long-context agents that supports three workflows—ingestion, retrieval, and maintenance—over a shared persistent state. Figure 3 shows the end-to-end system, and Figure 4 illustrates the core temporal index.

Design rationale. MemForest is designed to satisfy three requirements of long-horizon agent memory under continuous writes. First, new sessions should be incorporated incrementally with low write-path latency, so that memory remains fresh and queryable without introducing a serialization bottleneck on the update path. Second, updates should preserve temporally evolving state rather than collapsing memory into a single latest-state summary. The resulting memory must remain queryable not only for current-state lookup, but also for historical-state and temporal reasoning queries. Third, the write path should avoid any global memory object or global all-fact maintenance step. Each update should modify only the affected region of memory, so that maintenance cost depends on the size of new evidence rather than the size of accumulated state.

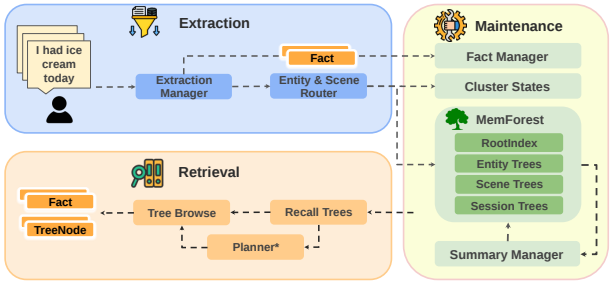

Figure 3: MemForest overview. New sessions are converted into canonical facts, materialized into scoped MemTrees, and queried through forest recall followed by tree browse. Maintenance edits only affected persistent state and selectively refreshes derived artifacts such as summaries, embeddings, and RootIndex rows. Optional modules are marked with ∗.

MemForest addresses these requirements through three design choices. It uses canonical facts as stable write units, separates persistent state from derived access artifacts, and materializes scoped memory as temporal indexes that support localized maintenance and coarse-to-fine retrieval.

3.1 Shared Memory Substrate

The core abstraction of MemForest is a shared memory substrate with two layers: *persistent state* and *derived artifacts*. Persistent state stores the durable semantics of memory and serves as the source of truth. In MemForest, it includes canonical facts, fact-to-tree mappings, cluster states, and the scoped tree structures themselves. Derived artifacts are auxiliary access structures regenerated from persistent state to accelerate retrieval and maintenance, including interval summaries, node embeddings, and root-level index rows.

This separation is central to the system design. Writes modify only affected persistent regions, while dependent derived artifacts are refreshed selectively rather than through full-state rewriting. As a result, the same substrate can support incremental writes, localized maintenance, and later lifecycle operations such as merge or deletion.

3.2 MemTree as a Scoped Temporal Index

MemTree is the core storage abstraction of MemForest. Each materialized scope is represented as a balanced temporal tree whose leaves preserve fine-grained evidence in chronological order, whose internal nodes summarize contiguous time intervals, and whose root provides a coarse representation for tree-level recall. Unlike

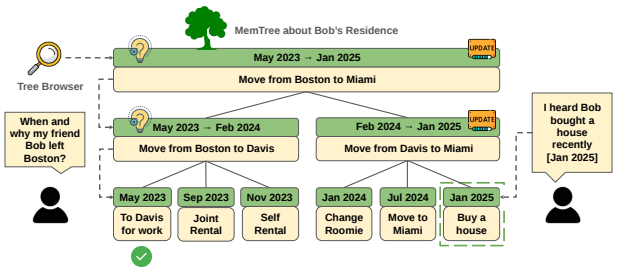

Figure 4: MemTree. Leaves preserve time-ordered evidence, internal nodes summarize contiguous intervals, and the root supports coarse tree-level recall. The same structure supports both query-time browse and localized refresh after writes.

flat memory stores, MemTree preserves state evolution directly in the index: older states remain accessible as earlier leaves and intervals rather than being overwritten by a latest-state summary.

MemTree serves two roles at once. On the write path, updates affect only touched leaves and ancestor regions, enabling localized maintenance. At query time, the same structure supports coarse-to-fine retrieval, allowing search to begin at root or interval summaries and descend only where finer evidence is needed.

MemForest materializes three complementary tree families. Session trees preserve dialogue order and local fidelity. Scene trees provide broader semantic access through clustered latent topics. Entity trees provide direct access to recurring subjects that accumulate evidence over time. Together, they expose dialogue-, topic-, and entity-scoped access paths over the same persistent substrate, so that retrieval can choose the scope that best matches a query rather than being tied to a single global index.

3.3 Workflow Overview

Ingestion workflow. Ingestion avoids coupling each new session to a shared global object. Raw interaction streams are processed chunk-locally, consolidated into canonical facts, routed into session-, entity-, and scene-level scopes, and materialized into the corresponding MemTrees. Because a new session is decomposed into stable write units and inserted only into affected scopes, write cost tracks the incoming update rather than the size of accumulated memory.

Retrieval workflow. Retrieval separates coarse pruning from fine-grained evidence search so that LLM cost is spent only where it improves answer quality. MemForest first performs forest-level recall to select a small set of candidate trees, then browses within those trees to locate answer-bearing evidence. This two-stage design keeps semantic and temporal grouping intact during coarse pruning while allowing evidence search to descend from root and interval summaries to leaf-level facts or dialogue units only where needed.

Tree browse supports two operating modes under the same retrieval stack: a lightweight embedding-only mode suitable for latency-sensitive settings, and an LLM-guided mode that supports higher-accuracy evidence localization when additional traversal cost is acceptable. In the default high-accuracy configuration, MemForest further uses a planner to generate targeted per-tree subqueries based on recalled root summaries; Section 6.2 analyzes the effect of planner use and browse policy.

Maintenance workflow. Maintenance reuses the ingestion-side locality principle so that incremental updates, merge, deletion, and related lifecycle operations remain bounded by the affected state rather than the full accumulated memory. Rather than rewriting a global memory object, MemForest edits only affected persistent regions and then selectively refreshes dependent derived artifacts, so that maintenance cost scales with touched scopes and access paths.

4 IMPLEMENTATION DETAILS

Section 3 described the overall MemForest architecture. We now present the implementation of its core mechanisms.

4.1 Extraction Workflow

The extraction workflow converts raw interaction streams into canonical facts and materializes them into scoped MemTrees through four stages: parallel extraction, canonicalization, routing, and tree materialization.

Parallel extraction. Given a session

$$D = (u_1, u_2, \ldots, u_T), \tag{2}$$

MemForest partitions it into fixed-size chunks

$$C(D) = \{c_i\}_{i=1}^{\lceil T/b \rceil}, \qquad c_i = (u_{(i-1)b+1}, \ldots, u_{\min(ib,T)}). \tag{3}$$

Chunks are processed independently up to the concurrency budget, exposing inference parallelism and avoiding the long serialized extraction path of profile-centric systems. This creates a latency–token trade-off: smaller chunks expose more parallel requests and bound each generation, but repeat prompt/schema input and JSON scaffolding; larger chunks amortize that overhead but reduce parallelism and can lose answer-bearing evidence as context grows [18]. For short sessions or below serving saturation, repeated prefill can outweigh the parallel-latency benefit.

Canonicalization. Because chunk-local extraction fragments memory, MemForest canonicalizes the outputs before indexing. Each canonical fact contains retrieval-ready fact text, temporal anchors, entity labels, and scene labels, and is stored in the Fact Manager as the system's stable write unit. Canonicalization uses normalized exact matching, embedding-retrieved candidates, and a bounded LLM equivalence check. Equivalent records merge provenance and occurrence times; contradictory or otherwise non-equivalent updates remain separate facts with time anchors. Canonicalization consolidates records but does not verify their truth.

Routing. Canonical facts are then routed into session, entity, and scene scopes. Session scope comes directly from the source dialogue. Entity scope is induced from normalized entity labels. Scene scope is induced by clustering over canonical facts and maintained with cluster states containing centroids and member-fact identifiers. A fact may enter multiple scopes; multi-assignment is not resolved by forcing one winning tree. Scene assignment uses cosine similarity to active cluster centroids, creates a new scene below the activation

threshold, and merges clusters only above the configured merge threshold. Session and scene paths remain available when an entity overlay is not activated. This stage remains lightweight and requires no additional LLM calls after extraction.

Tree materialization. Finally, routed facts are inserted into the corresponding MemTrees. Entity and scene trees use atomic facts as leaves, while session trees use original dialogue units to preserve a high-fidelity channel. Instead of rewriting a compact global state, MemForest inserts only the new evidence into affected scopes.

4.2 Retrieval Workflow

At query time, MemForest separates retrieval into *forest recall* and *tree browse*. Forest recall selects candidate trees; tree browse locates answer-bearing evidence inside them.

Forest recall. Given a query q, MemForest first recalls candidate trees from the forest. Root summaries provide a natural coarse retrieval unit, but may lose localized lexical cues. Atomic facts often preserve stronger signals for entity names, dates, and short event descriptions. MemForest therefore uses *union recall*.

First, we retrieve the top-k trees using root-summary embeddings:

$$C_{\text{root}}(q) = \text{TopK}\big(\text{sim}(e(q), e(r_T))\big)_{T \in \mathcal{F}}, \tag{4}$$

where $e(q)$ is the query embedding, $e(r_T)$ is the embedding of tree T's root summary, and $\mathcal{F}$ is the forest. All reported similarities are cosine similarities over normalized Qwen3 embeddings. The evaluated per-user RootIndex and NodeIndex use exact search rather than an ANN approximation.

Second, we retrieve the top-k atomic facts,

$$R_{\text{fact}}(q) = \text{TopK}_f\big(\text{sim}(e(q), e(f))\big), \tag{5}$$

map each fact back to the trees in which it appears, and collect

$$C_{\text{fact}}(q) = \bigcup_{f \in R_{\text{fact}}(q)} \text{Trees}(f), \tag{6}$$

where $\text{Trees}(f)$ denotes the set of trees containing fact f.

We then rank trees in the union pool by

$$\text{score}(q, T) = \max\big(\text{sim}(e(q), e(r_T)),\ s_{\text{fact}}(q, T)\big), \tag{7}$$

where

$$s_{\text{fact}}(q, T) = \begin{cases} \max\limits_{f \in T \cap R_{\text{fact}}(q)} \text{sim}(e(q), e(f)) & \text{if } T \cap R_{\text{fact}}(q) \neq \emptyset, \\ -\infty & \text{otherwise.} \end{cases} \tag{8}$$

The final recalled tree set is

$$C(q) = \text{TopK}_{T \in C_{\text{root}}(q) \cup C_{\text{fact}}(q)}\ \text{score}(q, T). \tag{9}$$

Root recall captures scope-level relevance, while fact-to-tree recall recovers trees whose relevance is concentrated in local evidence.

Tree browse. MemForest then browses within the recalled trees by descending from root and interval summaries to finer-grained branches until reaching leaf evidence. The retrieved leaves are resolved back to canonical facts or dialogue units, reranked, and assembled into the final answer context.

This browse stage supports two modes. In embedding-only mode, a promising child is one of the highest cosine-scoring child summaries. In LLM-guided mode, the browser presents child summaries and a tree-specific query to the branch-selection prompt, which selects the next child identifiers. Both modes share the same recalled trees and MemTree structure; they differ only in branch-selection policy.

Planner-guided per-tree subqueries. In the high-accuracy setting, tree browse can be paired with a planner. Given the original query and the recalled root summaries, the planner generates one targeted subquery per tree. This helps because different recalled trees often correspond to different aspects of the question, and the root summary provides a compact tree-level description.

The planner is therefore a traversal controller rather than an answer generator. Without it, the original query q is used for all trees. With it, browse proceeds using specialized per-tree subqueries. Section 6.2 studies its effect on retrieval quality and efficiency.

4.3 Maintenance Workflow

The main systems idea of MemForest is to separate *eager structural maintenance* from *lazy semantic refresh*. Structural edits are applied immediately, while summaries and other derived artifacts are refreshed later through dirty-node batching.

Eager structural maintenance. When new evidence arrives, MemForest routes each record into its affected trees, creates new leaves, updates placement maps, and performs any required insertion, split, or rebalance. Under the bounded-fan-out balancing policy, a k-ary tree with N leaves touches only the new leaf and its ancestor path, so structural maintenance affects

$$O(\log_k N) \tag{10}$$

nodes per affected tree. This is a conditional structural bound, not an end-to-end construction bound. The batch builder and online `auto_update` path report completion only after dirty summaries have been refreshed.

Lazy semantic refresh. After each structural edit, MemForest marks affected ancestors as dirty. Dirty nodes are collected, deduplicated, grouped by level, and refreshed bottom-up. Entity and scene leaves can pass through their canonical fact payload directly, whereas session leaves summarize raw dialogue units. Internal nodes summarize their children only after lower levels have been refreshed. Because multiple writes may share ancestors, each dirty node is summarized at most once per flush. Refresh is scheduled level-parallel, and nodes from different trees can be batched together. Algorithm 1 summarizes the process. Summary regeneration is the LLM-based maintenance step: a parent waits for its refreshed children, while same-level nodes and nodes in different trees form independent request waves.

Here "lazy" means deferral within a write batch: records are grouped by touched tree, dirty trees and ancestors are deduplicated, and each touched tree is refreshed once before the write completes.

If sessions are written concurrently, a tree-lock conflict requires both writes to touch the same tree. Table 2 shows that 83.4% of session pairs share no specific entity/scene tree. Facts from the same batch that map to one tree are coalesced into one insertion and one dirty-tree refresh rather than competing writers. The global user fallback is the only broad hotspot; it can be partitioned into

Table 2: Cross-session tree-write overlap (147 LongMemEval sessions).

Target trees	Trees/session	Shared	Disjoint
Specific entity/scene	6.65	16.6%	83.4%
Including user fallback	7.60	89.3%	10.7%

Shared is the fraction of within-user session pairs with at least one common target tree; the second row includes `entity:user`.

Algorithm 1: MemTree maintenance with eager structure and lazy refresh

Input : forest state $\mathcal{M}$, incoming canonical facts $\mathcal{F}$, incoming dialogue units $\mathcal{S}$
Output : updated persistent state and refreshed derived artifacts

$\mathcal{B} \leftarrow \text{RouteAndActivate}(\mathcal{M}, \mathcal{F}, \mathcal{S})$
foreach $(T, R) \in \mathcal{B}$ **do**
  foreach $r \in \text{SortByTime}(R)$ **do**
    $\ell \leftarrow \text{CreateLeaf}(r)$
    $\text{AttachAndRebalance}(T, \ell, r.time)$
    $\text{UpdatePlacementMap}(r, T, \ell)$
    $\text{MarkDirtyAncestors}(\ell)$
$\mathcal{T}_{\text{aff}} \leftarrow \{T \mid (T, R) \in \mathcal{B}\}$
$\mathcal{D} \leftarrow \text{CollectDirtyNodesByLevel}(\mathcal{T}_{\text{aff}})$
for $\lambda \leftarrow 0$ **to** $\text{MaxLevel}(\mathcal{D})$ **do**
  foreach $u \in \mathcal{D}[\lambda]$ ***in parallel*** **do**
    if $\lambda = 0 \wedge \text{TreeType}(u) \in \{\textit{ENTITY}, \textit{SCENE}\}$ **then**
      $u.summary \leftarrow \text{Passthrough}(u.payload)$
    else if $\lambda = 0$ **then**
      $u.summary \leftarrow \text{SummarizeCellText}(u.payload)$
    else
      $u.summary \leftarrow \text{SummarizeChildren}(u.children)$
foreach $u \in \text{AffectedNonLeafNodes}(\mathcal{T}_{\text{aff}})$ ***in parallel*** **do**
  $u.embedding \leftarrow \text{Embed}(u.summary)$
$\text{RefreshRootRows}(\mathcal{M}, \mathcal{T}_{\text{aff}})$
return $\mathcal{M}$

independently built subforests and consolidated through the migration/merge path in Section 5.6.

Lifecycle operations. The same local-update principle extends to other lifecycle operations.

Merge. Canonical-fact merge requires semantic equivalence and combines provenance and occurrence times; non-equivalent state updates remain distinct. Tree rebalance follows the bounded-fan-out invariant. MemForest then reconciles cluster states and the corresponding trees. If two scopes align to the same merged tree, only their touched subtrees are merged; otherwise unmatched trees can be copied directly.

Delete. If a dialogue segment is retracted, MemForest removes the corresponding facts and tree nodes, updates the mappings, and refreshes only invalidated summaries and embeddings.

Migration. External memory can be imported as another forest and integrated through the same merge path. Migration is therefore not a separate primitive, but a structured extension of merge.

4.4 Cost Rationale and Update Locality

MemForest shares the high-level intuition of write-optimized structures such as LSM-trees [23]: expensive maintenance is deferred and batched rather than triggered as repeated full-state rewrites. The deferred work here is semantic refresh, and we next quantify how this keeps maintenance cost bound to local updates.

Let F be the number of incoming canonical facts, U the maximum number of trees touched by one fact, N the maximum number of leaves in a touched tree, and k the branching factor. Across the batch, structural maintenance touches one root-to-leaf path per fact–tree assignment:

$$T_{\text{struct}} = O(FU \log_k N). \tag{11}$$

Across a batch, let D be the number of distinct dirty nodes after ancestor deduplication. Summary regeneration depends on D, not on total memory size:

$$T_{\text{refresh}} \propto \sum_{\lambda=0}^{h} |D_\lambda|, \qquad D = \bigcup_{\lambda=0}^{h} D_\lambda, \tag{12}$$

where D_λ is the dirty-node set at level λ and $h = \lceil \log_k N \rceil$. Thus total refresh work is $O(D)$, while level-parallel execution has at most $O(\log_k N)$ dependent refresh waves within a touched tree.

The key claim is therefore locality: semantic maintenance is bounded by affected paths and distinct dirty nodes rather than by the full accumulated memory state. End-to-end construction is not logarithmic. Under bounded fan-out and balanced MemTrees, only structural insertion and the dependency depth of a dirty path are height-bounded; extraction still scales with incoming content, and total refresh work scales with the distinct dirty nodes.

4.5 Tree-Level Concurrency and Snapshot Semantics

In the online asynchronous implementation, each MemTree owns an independent reader–writer lock. Insertion and summary refresh take the write lock of only the touched tree; operations on distinct trees are scheduled concurrently, while same-tree writes serialize. Range queries, browse primitives, and snapshot export use shared read locks, so concurrent reads proceed without conflicting with one another and wait only for a writer on the same tree. Short fact-store and root index updates use separate component locks. Under the online `auto_update` path, completion means that dirty summaries and root vectors have been refreshed. Snapshot save exports read-locked tree states to a temporary directory and renames the completed directory into place. Extraction errors may propagate into multiple scopes and summaries; provenance supports targeted deletion and refresh.

5 EVALUATION

We now evaluate MemForest as an end-to-end long-context memory system. Throughout this section, we use *write path* to refer to the extraction and maintenance pipeline that transforms incoming dialogue history into queryable persistent memory. Our evaluation studies three questions: (1) how effective MemForest is on long-context conversational memory benchmarks, (2) how the two retrieval operating modes of MemForest compare in answer quality,

and (3) how much the system reduces write-path and query-time cost relative to representative baselines.

5.1 Experimental Setup

All systems first transform each benchmark history into persistent memory through their native write paths. **MemForest-Planner** uses union recall and planner-guided tree browsing; **MemForest-Embed** uses the same memory but embedding-only descent.

We evaluate `Qwen3-4B`, `Qwen3-30B-A3B`, and `Gemma-4-12B-IT` as generative backbones, with `Qwen3-Embedding-0.6B` fixed for retrieval [35, 39]. Models are served on dedicated H100 GPUs using vLLM 0.18.0 and FlashAttention [4].

LongMemEval-S contains 500 question-specific instances across six categories; LoCoMo contains 1,986 questions across single-hop, multi-hop, open-ended, temporal, and adversarial categories [20, 33]. We compare EverMemOS [13], LightMem [9], MemoryOS [17], MemPalace [22], and Mem0 [3]. Following Zhou et al. [42], we reproduce Zep Local from the open Graphiti temporal-graph core, Neo4j, and self-hosted generation and embedding services. This is a local Graphiti realization rather than Zep Cloud.

We report pass@1, native write rate, and query latency. We re-judge every frozen main-table answer with `deepseek-v4-flash` at temperature zero and the corresponding released public benchmark prompt. The LoCoMo headline follows the pinned Mem0 and EverMemOS runners and covers categories 1–4 (1,540 questions); category 5 is reported separately in the public complete version. Mem0 is rebuilt with source event timestamps, session-bounded LoCoMo writes, complete message coverage, and a global top-10 flat budget.

We preserve each baseline's native construction, retrieval, and answer interface. Flat systems use a final top-10 fact budget. MemForest retrieves ten native tree units before expansion, MemPalace retrieves ten sessions, and Zep uses its native per-evidence-class limit for heterogeneous graph objects. Equal-flat-fact retrieval is evaluated separately in Section 6.2. Pinned configuration and validation records are available in the public baseline guide and complete version.

5.2 Main Result on LongMemEval

Table 3 reports the aligned public-prompt results.

MemForest-Planner reaches the highest overall score in both Qwen settings: 72.60% and 81.80%. Under Gemma, MemForest-Embed reaches 78.40%, within 0.4 points of the strongest row. Category leaders vary across backbones and question types. Section 5.4 compares quality with memory-build rate.

Table 3: LongMemEval-S pass@1 under the aligned public judge. SS = single-session; K-Upd. = knowledge update.

Method	Overall	SS-User	SS-Pref.	SS-Asst.	K-Upd.	Multi	Temp.
Qwen3-4B-Instruct-2507							
MemForest-Planner	**72.60**	**95.71**	70.00	83.93	70.51	59.40	**70.68**
MemForest-Embed	69.40	88.57	53.33	83.93	60.26	60.90	**70.68**
EverMemOS	61.20	84.29	66.67	69.64	64.10	55.64	48.12
LightMem	68.80	94.29	**76.67**	35.71	**79.49**	**65.41**	64.66
MemoryOS	64.20	87.14	46.67	89.29	50.00	57.14	60.90
MemPalace	60.00	91.43	36.67	33.93	65.38	63.91	52.63
Mem0	44.80	82.86	33.33	26.79	43.59	42.11	38.35
Zep Local	69.60	94.29	53.33	**98.21**	75.64	59.40	54.89
Qwen3-30B-A3B-Instruct-2507							
MemForest-Planner	**81.80**	**97.14**	83.33	83.93	75.64	**76.69**	**81.20**
MemForest-Embed	78.40	94.29	80.00	83.93	69.23	72.93	78.20
EverMemOS	67.00	91.43	53.33	75.00	79.49	62.41	51.13
LightMem	70.20	95.71	**86.67**	37.50	79.49	64.66	66.92
MemoryOS	63.80	87.14	60.00	89.29	62.82	52.63	53.38
MemPalace	53.60	81.43	50.00	35.71	62.82	52.63	42.86
Mem0	50.20	91.43	46.67	26.79	60.26	42.86	40.60
Zep Local	71.00	95.71	63.33	**96.43**	**82.05**	56.39	57.14
Gemma-4-12B-IT							
MemForest-Planner	77.80	95.71	63.33	46.43	82.05	74.44	85.71
MemForest-Embed	**78.40**	94.29	60.00	46.43	84.62	74.44	**87.97**
EverMemOS	78.00	95.71	83.33	76.79	84.62	72.93	69.17
LightMem	77.00	**97.14**	**86.67**	37.50	80.77	**75.19**	80.45
MemoryOS	66.00	95.71	70.00	94.64	66.67	47.37	55.64
MemPalace	72.80	88.57	50.00	94.64	79.49	60.90	68.42
Mem0	59.20	91.43	50.00	23.21	74.36	54.89	54.89
Zep Local	75.60	94.29	66.67	**96.43**	**89.74**	63.16	63.16

5.3 Main Result on LoCoMo

5.4 Write-Path Efficiency

Table 5 reports isolated native Qwen3-30B writes. Each method runs alone for one LongMemEval instance and one matched LoCoMo conversation; source-turn rates are comparable only within a benchmark. MemForest is 6.0× EverMemOS on LongMemEval and 9.5× on the matched LoCoMo conversation.

Figure 8 separates extraction, canonicalization, routing, structural insertion, dirty refresh, index update, and persistence. The two purely autoregressive stages, extraction and dirty refresh, account for 83.58% of measured build time. Adding hybrid canonicalization raises the LLM-involved share to 90.57%. Structural insertion, index update, and persistence together remain below 1%. An insertion marks the summaries of affected parents and ancestors dirty: a parent depends on its refreshed children, but nodes at the same level or in different trees do not depend on one another. Using the exact stage tokens and historical single-request throughput under the matched Qwen/vLLM configuration, the throughput-calibrated serial replay takes 54.58 minutes for extraction and 9.88 minutes for dirty refresh. The measured parallel stages take 122.31 and 27.92 seconds, yielding 26.8× and 21.2× speedups, respectively. In the evaluated released paths, Mem0's stateful add couples candidate retrieval, LLM reconciliation, and an ordered commit, while MemoryOS's triggered promotion and profile maintenance depend on selected same-user pages and the current profile. The public complete version reports the replay model, sensitivity range, calls, token/request diagnostics, and released-path dependencies for every method.

5.5 Query-Time Latency

We next evaluate **query-time latency**, the online cost of retrieving memory and producing an answer after memory has been constructed. This is a secondary but practical metric alongside write-path efficiency.

Table 6 summarizes retrieval time, answer generation time, and total latency under both 30B and 4B answer backbones.

Table 4: LoCoMo categories 1–4 pass@1 under the aligned public judge.

Method	Overall	Single-Hop	Multi-Hop	Open-Ended	Temporal
Qwen3-4B-Instruct-2507					
MemForest-Planner	78.12	80.86	81.21	53.12	75.70
MemForest-Embed	76.62	79.43	**81.56**	**56.25**	71.03
EverMemOS	**79.22**	**83.71**	79.79	47.92	**76.32**
LightMem	66.17	69.68	68.79	40.62	62.31
MemoryOS	65.45	71.70	71.99	39.58	51.09
MemPalace	51.56	55.77	68.79	29.17	32.09
Mem0	47.86	45.66	52.84	39.58	51.71
Zep Local	68.70	78.36	62.77	34.38	58.88
Qwen3-30B-A3B-Instruct-2507					
MemForest-Planner	84.09	84.66	**86.88**	**67.71**	85.05
MemForest-Embed	80.58	82.88	85.11	59.38	76.95
EverMemOS	**84.22**	**87.63**	84.04	51.04	**85.36**
LightMem	60.52	62.07	62.06	44.79	59.81
MemoryOS	68.31	75.98	72.34	50.00	50.16
MemPalace	55.84	59.81	72.70	33.33	37.38
Mem0	59.22	58.86	61.35	58.33	58.57
Zep Local	76.75	82.52	76.95	56.25	67.60
Gemma-4-12B-IT					
MemForest-Planner	82.66	80.98	89.36	57.29	88.79
MemForest-Embed	81.69	80.38	87.23	57.29	87.54
EverMemOS	**88.57**	**90.84**	**89.72**	**63.54**	**89.10**
LightMem	78.90	81.33	79.43	50.00	80.69
MemoryOS	78.51	80.98	83.33	57.29	74.14
MemPalace	55.71	58.98	77.30	28.12	36.45
Mem0	48.38	44.71	57.09	39.58	52.96
Zep Local	69.55	74.91	68.79	26.04	69.16

Table 5: Isolated native memory-build rate (source turns/s).

	LongMemEval-S		LoCoMo	
Method	Rate	vs. Zep	Rate	vs. Zep
MemForest	**2.841**	**24.9×**	**7.020**	**32.5×**
EverMemOS	0.471	4.1×	0.738	3.4×
Mem0	1.397	12.3×	0.586	2.7×
MemoryOS	0.202	1.8×	0.245	1.1×
Zep Local	0.114	1.0×	0.216	1.0×

Table 6: Query-time latency breakdown (seconds).

	30B setting			4B setting		
Method	Retrieval	Answer	Total	Retrieval	Answer	Total
Mem0	0.027	0.196	0.22	0.030	0.261	0.29
MemoryOS	0.442	1.846	2.29	0.492	0.611	1.10
EverMemOS	2.254	6.232	8.49	2.106	8.860	10.97
MemForest-Embed	0.542	1.645	2.19	0.474	1.949	2.42
MemForest-Planner	2.600	2.031	4.60	2.400	1.948	4.30

Overall latency. MemForest exposes a clear *latency–accuracy trade-off* through its two retrieval modes. The embedding-only variant achieves **2.19s** (30B) and **2.42s** (4B) total latency, comparable to MemoryOS and substantially faster than EverMemOS. The LLM-guided variant increases total latency to **4.60s** and **4.30s**, reflecting the additional reasoning cost during tree browsing.

Retrieval vs. answer cost. The breakdown shows that the main difference between the two MemForest variants lies in **retrieval time**, not answer generation. MemForest-Planner spends **2.4–2.6s** on retrieval, compared to only **0.47–0.54s** for MemForest-Embed, while answer time remains relatively stable across the two variants. This indicates that planner-guided browsing primarily adds cost on the retrieval side, while leaving the downstream answer generation stage largely unchanged.

Comparison with baselines. Compared with EverMemOS, MemForest reduces query latency substantially (e.g., **4.60s vs. 8.49s** under 30B). Part of EverMemOS's larger answer-side latency comes from its *self-judge* style answer workflow, which introduces extra reasoning overhead after retrieval. By contrast, MemForest keeps the online path simpler once evidence has been assembled. Compared with MemoryOS, MemForest-Embed achieves similar latency with higher accuracy (Sections 5.2 and 5.3), indicating a more favorable quality–latency balance. Mem0 remains the fastest system because its retrieval pipeline is minimal, but this comes with a large drop in answer quality.

Takeaway. Absolute query-time latency is small across all compared systems relative to memory-construction cost, so online query overhead is unlikely to be the main bottleneck in practice. MemForest's systems advantage therefore remains its write-efficient memory construction, while the two browse modes offer two practical operating points: a lower-latency embedding-only mode and a higher-accuracy LLM-guided mode.

5.6 Efficient Migration

Beyond the initial write-path, a practical persistent memory system should also support *maintenance after construction*, including migration and merge. We therefore build a migration workload from LongMemEval by progressively combining multiple question instances into one memory state. The merged instances come from different LongMemEval users; we treat the resulting state as a synthetic stress workload that simulates a shared or multi-source memory store, rather than a single user's accumulating history. We compare *sequential write*, which linearly rewrites all sessions into a growing memory, with *migration merge*, which directly merges already materialized MemForest states. Both strategies are evaluated on memory scale and on the wall-clock cost of producing the merged state; the question-answering experiments in this paper are not replayed against the merged state, so we do not report post-merge answer accuracy here.

Figure 5 shows that migration merge is consistently faster. As the number of merged instances grows, migration exceeds **2×** speedup from N=3 onward, peaks at **2.70×** around N=5–6, and remains above **2.4×** at N=8. The gain comes from reuse: migration avoids repeated extraction over already processed histories and avoids rebuilding unaffected trees and subtrees from scratch.

Table 7 shows that this speedup does not come with obvious memory degradation. Sequential write and migration merge produce memory states of comparable scale, without evidence of systematic blow-up: across all N the numbers of facts differ by less than 1% and the numbers of trees by at most about 8% (e.g., 1,362 vs. 1,472 trees at N=8). Small differences are expected because fact extraction and tree-summary refresh are both LLM-based and

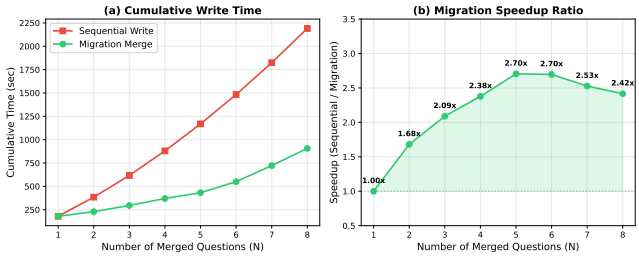

Figure 5: Migration efficiency on progressively merged LongMemEval instances. Left: cumulative maintenance time under sequential write and migration merge. Right: speedup of migration relative to sequential write.

Table 7: Memory scale under sequential write and migration merge. The two strategies produce memory states of comparable scale, without evidence of systematic blow-up.

N	Seq Facts	Mig Facts	Seq Trees	Mig Trees
1	1,435	1,435	249	249
2	2,692	2,716	375	405
3	4,061	4,098	562	607
4	5,540	5,590	733	792
5	6,860	6,922	895	967
6	8,101	8,174	1,044	1,128
7	9,426	9,511	1,200	1,296
8	10,705	10,801	1,362	1,472

therefore stochastic, and tree counts are additionally sensitive to scope-routing decisions during merge.

This experiment evaluates direct merge across already materialized memory states. MemForest exposes this interface, whereas the compared baseline adapters accept continued writes of source histories. The operation applies to memory transfer and synchronization across instances, including expert memory transfer [28] and distributed memory construction [12, 16].

Overall, the migration results extend the systems argument of the paper: MemForest is efficient not only on the initial write path, but also on post-build maintenance that reuses already materialized memory state.

6 ABLATION STUDY

The main results show that MemForest improves both answer quality and write-path efficiency. We now ask a more targeted question: which design choices are necessary for these gains? We organize the ablations around the two central design choices of MemForest. First, we study write-path scalability, focusing on the MemTree mechanisms that make maintenance local and scalable. Second, we study retrieval accuracy on a controlled LongMemEval subset, isolating the role of temporal tree views and selective browse policies.

6.1 Write-Path Scalability

We first study the write path, since MemForest's main systems contribution is not merely faster extraction, but the ability to keep long-horizon memory maintenance local as memory grows. The key question is therefore whether MemTree maintenance remains scalable under increasing tree size, and whether moderate fan-out is necessary to balance efficiency and summary quality.

Lazy maintenance reduces redundant LLM work and improves scalability. Figure 6(a) compares eager per-insert rewriting against MemForest's batch mark-dirty update strategy. Batch refresh substantially reduces summary calls, and the reduction becomes larger as the number of facts per tree grows. This shows that lazy maintenance is not a small implementation optimization: it is what prevents repeated writes from triggering redundant LLM regeneration along overlapping ancestor paths.

The same locality benefit appears in parallel execution. Figure 6(c) shows that level-parallel flush yields larger speedups for larger trees, because bigger trees expose more same-level nodes that can be refreshed concurrently. In other words, the gain from MemTree maintenance improves with data scale rather than disappearing at larger workloads.

Moderate fan-out is necessary for both efficiency and summary quality. Figures 6(d) and 6(e) study the branching factor k. Increasing k makes trees shallower and can reduce maintenance depth, but it also overloads each summary call. Per-call summary recall remains high up to about $k \approx 16$, after which it drops more sharply, indicating a soft summary-capacity knee. End-to-end root recall peaks at moderate k values rather than at the extremes: small k deepens the summarization chain and accumulates cascade loss, while overly large k makes each summary too flat and lossy. These results justify the use of moderate fan-out defaults in MemForest rather than treating k as an arbitrary tuning knob.

Extraction remains the dominant cost, but MemTree maintenance does not become the bottleneck. Figure 6(b) summarizes per-question build profiling. Extraction still dominates both time and token usage, which identifies it as the main remaining optimization target. By contrast, tree building contributes only a modest fraction of total wall-clock time despite issuing the largest share of LLM calls, because most tree-level summaries are short. This is an important systems result: MemTree introduces structure and local maintenance without turning the temporal index itself into the new bottleneck.

Chunk size controls the latency–token trade-off. Figure 7 reports a controlled assembled-long-session sweep using Gold Coverage (answer-bearing gold-span retention), facts/s, tokens/fact, extracted-fact count, and total input-plus-output tokens. Larger chunks lower total token work chiefly by emitting fewer facts, which also lowers extraction fidelity. Gold Coverage falls and tokens/fact ultimately worsens as answer-bearing evidence is omitted, consistent with the documented tendency of long-context LLMs to underuse evidence in the middle of their inputs [18]. A practical rule is to choose the largest local window that preserves extraction fidelity while still exposing enough independent cells to saturate the serving backend. We select $b = 2$ because it preserves 100% Gold Coverage, remains

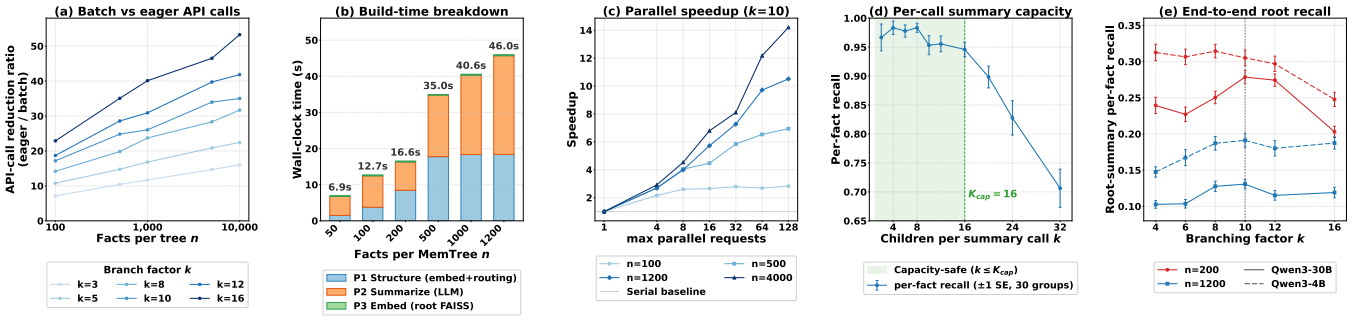

Figure 6: Write-path scalability diagnostics for MemTree. (a) Batch mark-dirty reduces LLM summary calls relative to eager per-insert rewriting. (b) Build-time breakdown shows that extraction dominates total write cost, while tree maintenance remains lightweight. (c) Level-parallel flush yields larger speedups on larger trees, showing improved scalability with data size. (d) Per-call summary capacity remains stable up to a moderate fan-out before dropping. (e) End-to-end root recall peaks at moderate branching factors, motivating the default k settings used in MemForest.

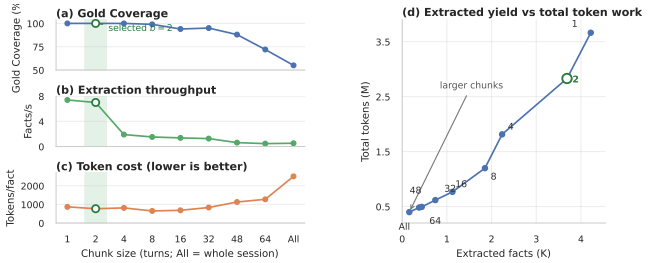

Figure 7: Chunk-size trade-off on the assembled-long-session diagnostic. (a)–(c) report Gold Coverage, throughput, and per-fact token cost; (d), whose point labels denote chunk size, relates extracted-fact yield to total token work. Coarse chunks lower aggregate work partly by omitting facts rather than preserving quality at lower cost.

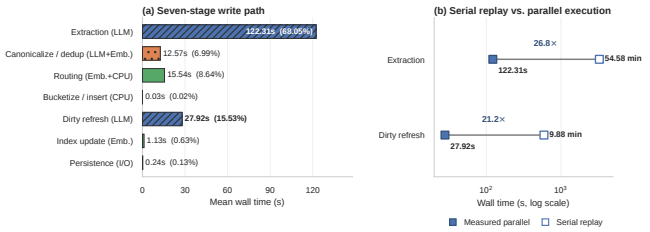

Figure 8: Detailed Qwen3-30B write-path profile, averaged over three representative LongMemEval streams (179.74 seconds total). (a) Each stage is labeled by its dominant workload. (b) Measured parallel wall time is compared with throughput-calibrated serial replay of the same LLM requests.

near the highest throughput, and uses fewer tokens per fact than $b = 1$.

Table 8: Held-out equal-budget gold-fact coverage (%).

Representation	k=10	k=30	k=50
Flat Log	79.1	86.4	89.0
Log-Time	**80.3**	87.3	91.0
Validity Interval	77.1	82.8	86.2
Scratchpad+Background	77.6	84.2	87.5
Triplet Graph	78.8	85.5	88.0
MemTree	79.6	**87.4**	**92.0**

6.2 Temporal Representation and Retrieval Controls

We manually map 319 of the 321 official LoCoMo temporal questions to their answer-supporting canonical facts; two questions without reliable mappings are excluded. We reserve 82 mappings exclusively for selector development and report the remaining 237 held-out questions. Flat Log, tuned Log-Time, Validity Interval, Scratchpad+Background, Triplet Graph, and MemTree use the same canonical facts, timestamps, embeddings, and final fact budgets. Macro gold-fact coverage first computes the fraction of mapped gold facts retrieved for each question and then averages across questions. Log-Time and the lightweight MemTree selector each select their weights on the same development split and freeze them before held-out evaluation.

The control isolates representation and selection before answer generation; Tables 3 and 4 report the native end-to-end systems. Tuned Log-Time leads by 0.7 points at $k = 10$, and the two are effectively tied at $k = 30$. MemTree has the highest observed coverage at $k = 50$ (92.0%), supporting its main role as a structured high-recall candidate source when more evidence can be retained. Flat Log and Log-Time remain cheaper after shared extraction because they do not materialize hierarchical summaries. Full curves, confidence intervals, development grids, and candidate-pool diagnostics are in the public complete version.

Routing accuracy and scene stability. A reviewed audit of 127 facts with active entity overlays finds 124/127 (97.6%) semantically valid assignments. Only 9/127 cover every expected entity, so entity routing remains a selective overlay alongside session/scene scopes and fact-to-tree union recall. In 231 reviewed temporal mappings,

Table 9: Tree-family ablation on the 60-question LongMemEval subset. We report 30B pass@8 under `llm+planner` browse.

Variant	30B pass@8
entity+scene+session	**86.7**
entity+scene	**86.7**
entity+session	85.0
session only	85.0
scene+session	83.3
scene only	81.7
entity only	63.3

evidence spans multiple specific trees for 4/130 LoCoMo and 37/101 LongMemEval questions; under embedding browse, fragmented versus unfragmented accuracy is 50.0% versus 76.2% and 67.6% versus 78.1%, respectively. Perturbing the scene activation threshold around its default gives assignment-set Jaccard mean/p05 of at least 0.9997/0.9996 for 4B and 0.9650/0.8829 for 30B. Full labels are in the public complete version. These results support a high-precision but deliberately redundant routing design: active entity routes are usually valid, but no single overlay is complete and fragmentation remains material on LongMemEval. MemForest therefore retains multi-scope placement, fact-to-tree union recall, and multi-tree browse instead of committing each fact to one route.

We next study retrieval accuracy on a controlled 60-question LongMemEval subset, constructed by sampling 10 questions from each of the six question types. To reduce the influence of single-sample answer variance, we report pass@8 in the main text and provide the full question IDs in the public complete version. These experiments serve as retrieval diagnostics rather than replacements for the full-benchmark results in Section 5.2. We focus on two questions: whether multiple temporal tree views are necessary, and whether planner-guided tree browse improves evidence access once candidate trees have been recalled.

Structured views provide the measured gain; session trees preserve a fallback. Table 9 compares tree-family combinations under the same planner-guided LLM browse setting. Among single views, `session only` reaches **85.0%** pass@8, `scene only` reaches **81.7%**, and `entity only` reaches **63.3%**.

The `entity+scene` combination reaches **86.7%**, matching the full three-family setting on this subset. Thus entity and scene trees supply the measured structured-retrieval gain, while the session tree retains raw conversational evidence as a fallback; it adds no measured pass@8 gain on this subset.

Planner-guided LLM browse yields the highest measured accuracy. We next compare the six retrieval variants in Table 10 on the same 60-question subset. `flat-10` omits trees, `root-only` omits descent, `emb`/`llm` selects branches by embedding/LLM, and `+planner` adds per-tree subqueries. This comparison isolates how much the temporal tree structure and browse policy each contribute.

The first result is that neither flat retrieval nor root-only recall is sufficient. Flat top-10 retrieval reaches **78.3%**, while root-only recall is lower at **73.3%**. Together, these two baselines show that neither

Table 10: Retrieval and browse ablation on the 60-question LongMemEval subset. We report 30B pass@8 only.

Variant	30B pass@8
flat-10	78.3
root-only	73.3
emb	85.0
emb+planner	83.3
llm	88.3
llm+planner	**90.0**

directly retrieving a flat top-10 set nor stopping at coarse root summaries can match the retrieval power of temporal MemTree browse. The system must both preserve temporal scopes and descend within them to reach answer-bearing evidence.

The second result is that browse policy materially affects accuracy. Embedding-only browse already improves substantially over both flat and root-only baselines, reaching **85.0%**; pure LLM-guided browse reaches **88.3%**; and planner-guided LLM browse reaches the best result, **90.0%**. Once candidate trees have been recalled, planner-generated per-tree subqueries make browse more targeted by using each tree's root summary to specialize the search intent. This is especially useful in the LLM-guided setting, where the browser can exploit not only semantic similarity but also more structured intent such as temporal focus, state transitions, or finer-grained disambiguation. The added planner cost is acceptable here, because a single LLM call steers the subsequent per-tree LLM browse calls.

By contrast, planner guidance does not help embedding browse: planner-guided embedding browse slightly drops to **83.3%**. Under embedding-only descent, the rewritten query is still reduced to a vector similarity signal. Unlike LLM-guided branch selection, embedding similarity cannot reliably preserve richer browse intent such as time ranges, before/after relations, or transition constraints; it mostly captures semantic relatedness. In that setting, the extra planner call adds noticeable cost without a comparably targeted retrieval benefit. For this reason, the final system retains two operating modes: a lightweight embedding-only mode for latency-sensitive deployments, and a planner-guided LLM-guided mode for the high-accuracy setting.

7 RELATED WORK

Existing memory systems for LLM agents differ mainly in how memory is constructed, how persistent state is organized, and how that state is maintained as interactions accumulate. We review the work most related to MemForest along these three dimensions.

Memory Construction. A common direction is to build long-term memory by extracting salient information from interactions and storing it as persistent records. SeCom studies how memory units should be constructed and retrieved for conversational agents [25]. Mem0 emphasizes practical long-term memory for personalization and retrieval [3]. LightMem and MemoryOS introduce staged memory handling across short-, mid-, and long-term components, separating online use from offline consolidation and managed updates [9, 17]. Context-dependent memory frameworks highlight modular memory handling across interaction contexts [10].

MemForest focuses specifically on construction-path parallelism: it extracts short windows concurrently, consolidates them into canonical facts, and materializes indexes from that state.

Memory Organization. A second line of work studies how persistent memory should be organized after it is written. MemForest is aligned with this line, but differs in the representation it materializes: it organizes canonical facts into per-scope temporal trees so that retrieval can proceed from root summaries to leaf evidence within an explicit temporal hierarchy, rather than centering the design on recollection workflows or dynamically selected structures. EverMemOS models memory as a lifecycle of semantic consolidation and recollection [13]. A-Mem explores agentic memory organization, while H-Mem, HiMem, and Adapt investigate hybrid, hierarchical, or adaptive structures for long-context agents [15, 19, 34, 36, 38]. These systems move beyond flat memory stores and show that retrieval quality depends strongly on memory structure.

Existing systems accumulate temporal memory in several ways: MemPalace keeps timestamped raw events; LightMem and MemoryOS combine recent memory with background consolidation; and Zep/Graphiti stores bi-temporal graph edges with retained provenance [9, 17, 22, 27]. EverMemOS preserves event timestamps in its memory units and event logs. Its agentic search combines hybrid retrieval with an LLM sufficiency check and, when needed, a second serial query-refinement round [13]. It can therefore recover historical evidence through query-time LLM reasoning rather than requiring the index itself to expose an explicit temporal-navigation path. These representations can preserve historical evidence; they differ in write-time construction, maintenance, and query-time reconstruction. We therefore compare both shared-fact representation controls and a native Zep Local implementation in Sections 6.2 and 5.

Memory Maintenance. Another important question is how memory should evolve after it is written. MemForest differs from prior work by making a canonical, mergeable memory state with explicit temporal update semantics a first-class design goal: canonical facts are persistent state, summaries are derived artifacts regenerated incrementally, and this separation supports temporal preservation, incremental reorganization, and migration without replaying raw sessions. RMM treats memory as a reflective process, refining what is retained and how stored memory is later used [32]. At the other end of the design space, fidelity-first systems such as MemPalace preserve raw or near-verbatim history and defer abstraction to query time [22]. These approaches make different trade-offs between write-time processing and query-time reasoning, but neither centers on canonical, locally maintainable persistent state.

8 CONCLUSION

We presented MemForest, a persistent agent-memory system that uses canonical facts and scoped temporal trees to localize maintenance under continuous writes. Under the aligned public protocol, MemForest-Planner leads LongMemEval-S with both Qwen backbones (72.60% and 81.80%) and nearly ties EverMemOS on Qwen3-30B LoCoMo categories 1–4 (84.09% versus 84.22%). Gemma produces mixed rankings, indicating model-dependent answer quality. Isolated writes and stage profiling show that parallel extraction and level-wise dirty refresh shorten the wall-clock critical path, while extraction remains the dominant cost.

MemForest optimizes freshness latency rather than raw token count: parallel local requests repeat fixed prompt work but shorten wall time. Chunk size, concurrency, output caps, browse mode, and evidence budget expose practical operating points. Logs, intervals, graphs, and MemTrees remain valid choices with different construction, maintenance, and retrieval trade-offs.

REFERENCES

[1] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-Craft: Managing Chunk-Caches for Efficient Retrieval-Augmented Generation. *Proc. ACM Manag. Data* 3, 3, Article 136 (June 2025), 28 pages. doi:10.1145/3725273

[2] Bruno Becker, Stephan Gschwind, Thomas Ohler, Bernhard Seeger, and Peter Widmayer. 1996. An asymptotically optimal multiversion B-tree. *The VLDB Journal* 5, 4 (1996), 264–275.

[3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413* (2025).

[4] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*.

[5] DeepSeek. 2026. *Context Caching.* https://api-docs.deepseek.com/guides/kv_cache

[6] DeepSeek. 2026. *Models and Pricing.* DeepSeek. https://api-docs.deepseek.com/quick_start/pricing

[7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).

[8] Ramez Elmasri, Gene TJ Wuu, and Yeong-Joon Kim. 1990. The time index: An access structure for temporal data. In *Proceedings of the 16th International Conference on Very Large Data Bases*. 1–12.

[9] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. 2026. LightMem: Lightweight and Efficient Memory-Augmented Generation. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=dyJ0GWpjJB

[10] Pengyu Gao, Jinming Zhao, Xinyue Chen, and Long Yilin. 2025. An efficient context-dependent memory framework for llm-centric agents. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*. 1055–1069.

[11] Yubin Ge, Salvatore Romeo, Jason Cai, Raphael Shu, Yassine Benajiba, Monica Sunkara, and Yi Zhang. 2025. Tremu: Towards neuro-symbolic temporal reasoning for llm-agents with memory in multi-session dialogues. In *Findings of the Association for Computational Linguistics: ACL 2025*. 18974–18988.

[12] Tooraj Helmi. 2025. Decentralizing AI Memory: SHIMI, a Semantic Hierarchical Memory Index for Scalable Agent Reasoning. *arXiv preprint arXiv:2504.06135* (2025).

[13] Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, et al. 2026. EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning. *arXiv preprint arXiv:2601.02163* (2026).

[14] Guoyu Hu, Shaofeng Cai, Tien Tuan Anh Dinh, Zhongle Xie, Cong Yue, Gang Chen, and Beng Chin Ooi. 2025. HAKES: Scalable Vector Database for Embedding Search Service. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3049–3062.

[15] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. 2025. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 32779–32798.

[16] Paul Jackson and Jane Klobas. 2008. Transactive memory systems in organizations: Implications for knowledge directories. *Decision support systems* 44, 2 (2008), 409–424.

[17] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory os of ai agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 25972–25981.

[18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638

[19] Mingfei Lu, Mengjia Wu, Feng Liu, Jiawei Xu, Weikai Li, Haoyang Wang, Zhengdong Hu, Ying Ding, Yizhou Sun, Jie Lu, et al. 2026. Choosing How to Remember: Adaptive Memory Structures for LLM Agents. *arXiv preprint arXiv:2602.14038* (2026).

[20] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13851–13870.

[21] Mem0. 2026. Memory Benchmarks. https://github.com/mem0ai/memory-benchmarks. Judge prompts referenced by immutable repository commits.

[22] milla-jovovich. 2026. *MemPalace.* https://github.com/milla-jovovich/mempalace

[23] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The log-structured merge-tree (LSM-tree). *Acta informatica* 33, 4 (1996), 351–385.

[24] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. MemGPT: towards LLMs as operating systems. (2023).

[25] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Jianfeng Gao. 2025. SeCom: On Memory Construction and Retrieval for Personalized Conversational Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=xKDZAW0He3

[26] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.

[27] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956* (2025).

[28] Alireza Rezazadeh, Zichao Li, Ange Lou, Yuying Zhao, Wei Wei, and Yujia Bao. 2025. Collaborative memory: Multi-user memory sharing in llm agents with dynamic access control. *arXiv preprint arXiv:2505.18279* (2025).

[29] Alireza Rezazadeh, Zichao Li, Wei Wei, and Yujia Bao. 2025. From Isolated Conversations to Hierarchical Schemas: Dynamic Tree Memory Representation for LLMs. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=moXtEmCleY

[30] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. Raptor: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*.

[31] Ji Sun, Guoliang Li, James Pan, Jiang Wang, Yongqing Xie, Ruicheng Liu, and Wen Nie. 2025. GaussDB-Vector: A Large-Scale Persistent Real-Time Vector Database for LLM Applications. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4951–4963.

[32] Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, et al. 2025. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8416–8439.

[33] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=pZiyCaVuti

[34] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=FiM0M8gcct

[35] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[36] Zihe Ye, Jingyuan Huang, Weixin Chen, and Yongfeng Zhang. 2026. H-Mem: Hybrid Multi-Dimensional Memory Management for Long-Context Conversational Agents. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7756–7775.

[37] Runjie Yu, Weizhou Huang, Shuhan Bai, Jian Zhou, and Fei Wu. 2025. AquaPipe: A Quality-Aware Pipeline for Knowledge Retrieval and Large Language Models. *Proc. ACM Manag. Data* 3, 1, Article 11 (Feb. 2025), 26 pages. doi:10.1145/3709661

[38] Ningning Zhang, Xingxing Yang, Zhizhong Tan, Weiping Deng, and Wenyong Wang. 2026. HiMem: Hierarchical Long-Term Memory for LLM Long-Horizon Agents. *arXiv preprint arXiv:2601.06377* (2026).

[39] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176* (2025).

[40] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems* 43, 6 (2025), 1–47.

[41] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 19724–19731.

[42] Wei Zhou, Xuanhe Zhou, Shaokun Han, Hongming Xu, Guoliang Li, Zhiyu Li, Feiyu Xiong, and Fan Wu. 2026. Are We Ready For An Agent-Native Memory System? *arXiv preprint arXiv:2606.24775* (2026).

A PROMPTS

A.1 LLM-as-Judge Prompts

LongMemEval Judge Prompt

Your task is to label an answer to a LongMemEval question as CORRECT or WRONG.
You will be given:
(1) question type
(2) question
(3) gold answer
(4) generated answer
Be generous with grading: (1) accept semantically equivalent answers; (2) accept equivalent relative and absolute time expressions; (3) accept more specific but consistent answers; (4) mark WRONG only for contradiction, missing key facts, answering a different question, or incorrect abstention.
Question type: {question_type}
Question: {question}
Gold answer: {gold_answer}
Generated answer: {generated_answer}
Return JSON only with {"label": "CORRECT"} or {"label": "WRONG"}.

LoCoMo Judge Prompt

Your task is to label an answer to a question as CORRECT or WRONG.
You will be given:
(1) question
(2) gold answer
(3) generated answer
Be generous with grading: (1) accept semantically equivalent answers; (2) accept equivalent relative and absolute time expressions; (3) accept more specific but consistent answers; (4) mark WRONG only for contradiction, missing key facts, non-answer, or incorrect abstention.
Question: {question}
Gold answer: {gold_answer}
Generated answer: {generated_answer}
Return JSON only with {"label": "CORRECT"} or {"label": "WRONG"}.

A.2 Retrieval and Answer Interfaces

Flat-item systems in the main tables expose a global final top-$k = 10$ context. We preserve each method's schema-aware answer interface because the returned objects differ: MemForest returns canonical facts and tree-derived evidence, EverMemOS returns episodic/recollection contexts, and other flat systems expose their native memory records. MemForest uses its native top-$k = 10$ tree-browse setting and fully expands selected tree units; the equal-flat-fact comparison is confined to the controlled retrieval study. Zep Local is not described as an equal ten-fact comparison: its native Graphiti search uses a limit of 10 per evidence class and serializes heterogeneous edges, nodes, episodes, and communities. Table A1 reports the resulting object counts and context-token lengths separately. Frozen answers are judged with `deepseek-v4-flash`, temperature zero, and thinking disabled.

The pinned EverMemOS rows preserve event timestamps in memory units and event logs and use its agentic retrieval mode. Round one performs hybrid retrieval and an LLM sufficiency check; if evidence is insufficient, LLM-refined queries drive a second retrieval round before final reranking. The released configuration keeps the final response budget at ten memory units.

Following Zhou et al. [42], the Zep Local row uses the released Graphiti/Neo4j adapter: Graphiti v0.24.1, Neo4j 5.26.2, raw-episode ingestion, and self-hosted generation and Qwen3-Embedding-0.6B endpoints. It is a reproducible local Graphiti realization, not Zep Cloud. The six-cell budget summary, its source manifest, and the aggregation script preserve the table's result-to-code path.

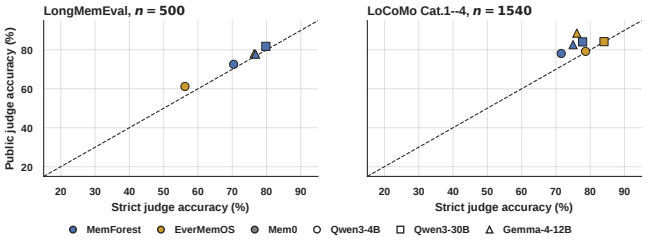

Figure A1: Paired strict/public-prompt sensitivity with frozen retrieval and answers. The diagonal denotes no change.

The native recipe therefore usually serializes approximately 20 heterogeneous objects, not ten flat facts. Small cross-backbone context-length differences arise because Graphiti extraction builds a different graph for each generative backbone.

We take both public prompts from the Mem0 benchmark repository [21]. LongMemEval uses root commit `7ba1bd3`. LoCoMo categories 1–4 use the tuned public prompt in commit `edcd6f1`. The headline scope follows the released public runners. At the pinned commit, the Mem0 harness uses the explicit setting

```
CATEGORIES_TO_EVALUATE = [1,2,3,4].
```

It marks the 446 category-5 questions as excluded from scoring and reports public headline results as 1,410/1,540 at top-200 and 1,273/1,540 at top-50.[1] The pinned EverMemOS runner likewise applies `filter_category:[5]`, and its reported LoCoMo evaluation contains 1,540 questions [13].[2] The public prompt assumes a non-empty reference answer, so it cannot define the policy for adversarial category 5; we retain the strict answerability judge for that category.

A.3 Strict/Public Judge Sensitivity

The paired diagnostic freezes retrieval and generated answers and changes only the judge prompt. It contains 3 methods ×3 votes ×[172 LongMemEval questions ×2 judge arms +200 LoCoMo questions ×3 judge arms] = 8,496 successful judge calls, with zero judge errors. The reported temporal slices contain 122 LongMemEval and 150 LoCoMo questions within those broader frozen samples. On LoCoMo temporal, the corresponding public evaluation prompt increases EverMemOS, MemForest, and Mem0 by 17.33, 16.00, and 5.33 points. On LongMemEval temporal, the increases are 4.92, 2.46, and 4.92 points. The effect varies by benchmark and method, including MemForest.

[1] https://github.com/mem0ai/memory-benchmarks/blob/edcd6f1d42400837b1fcb6997716f1769dc51a37/benchmarks/locomo/prompts.py; https://github.com/mem0ai/memory-benchmarks/blob/edcd6f1d42400837b1fcb6997716f1769dc51a37/README.md
[2] https://github.com/EverMind-AI/EverMemOS/blob/539db77d5cc804c875246e34611fd266bf8c1e5d/evaluation/config/datasets/locomo.yaml

Table A1: Zep Local native retrieval budget. Object columns report the mean number serialized per question. Context tokens use the exact serialized context and one fixed Qwen tokenizer; they exclude the answer instruction and question.

Backbone	Benchmark	Edges	Nodes	Episodes	Communities	Context tokens avg.	Context tokens p95
Qwen3-4B	LongMemEval-S	5.00	5.00	9.99	0.00	3,269	4,542
Qwen3-4B	LoCoMo	5.00	4.97	10.00	0.00	1,360	1,626
Qwen3-30B	LongMemEval-S	5.00	5.00	9.99	0.00	3,125	4,412
Qwen3-30B	LoCoMo	5.00	5.00	10.00	0.00	1,210	1,413
Gemma-4-12B-IT	LongMemEval-S	5.00	5.00	9.99	0.00	3,105	4,364
Gemma-4-12B-IT	LoCoMo	5.00	5.00	10.00	0.00	1,168	1,348

Table A2: Final evaluation protocol and sensitivity controls.

Component	Main setting	Sensitivity control	Reporting rule
Retrieval budget	Native top-10 units; MemForest expands tree units; Zep uses per-class limit 10	Equal flat-fact sweep; Mem0 occupancy at top-50/200	Report unit semantics and Zep object counts separately
Timestamp	Benchmark event time in timestamp and metadata; LoCoMo batches bounded by source session	Automated date-range and batch-coverage gates	Reject runtime dates and incomplete coverage
Answer prompt	Native schema-aware interface	Shared schema-neutral prompt	Treat shared-prompt scores as diagnostic
Judge	Benchmark-specific public prompts with `deepseek-v4-flash`	Strict paired judge	Main tables use the corresponding public prompt
LoCoMo Cat.5	Strict answerability judge	Not applicable	Report separately because the public prompt assumes non-empty gold

Table A3: LoCoMo category-5 strict answerability pass@1.

Method	Qwen3-4B	Qwen3-30B	Gemma-4-12B-IT
MemForest-Planner	29.60	35.87	10.09
MemForest-Embed	22.87	32.29	9.42
EverMemOS	23.77	19.73	8.97
LightMem	11.66	14.13	14.13
MemoryOS	**42.38**	**44.84**	**28.92**
MemPalace	14.57	15.92	16.59
Mem0	21.30	22.42	13.90
Zep Local	10.09	11.43	6.73

A.4 LoCoMo Adversarial Category

Category 5 evaluates answerability instead of supplying the non-empty reference answer assumed by the released public LLM-judge runners. The headline comparison follows the released scope of 1,540 questions; we evaluate the 446 category-5 questions separately with the same strict answerability prompt for every method. MemoryOS obtains the highest category-5 score under all three backbones.

A.5 Mem0 Budget Occupancy and Answer-Prompt Coupling

The corrected Qwen3-30B Mem0 snapshot contains 121,594 memory units across 500 isolated LongMemEval-S stores, or 243.2 units per question on average. Top-10 exposes 4.18%, top-50 exposes 20.92%, and top-200 exposes 83.11% of a local store on average; top-200 exhausts 40/500 stores. On temporal questions it exposes 82.48% and exhausts 8/133 stores. We therefore treat the public top-200 result as a different-budget, broad high-coverage setting at this memory scale, not as a direct selective-retrieval comparison. The exact store-level occupancy distribution is released with the corrected Mem0 results.

Forcing one schema-neutral answer prompt changes schema interpretation, abstention, and evidence use because methods serialize heterogeneous objects differently. We therefore retain native schema-aware answer interfaces in the main protocol and use equal-budget retrieval coverage, rather than shared-prompt QA, as the representation-level diagnostic.

B TEMPORAL RETRIEVAL CONTROLS

B.1 Temporal Diagnostic Subset Map

Several temporal subsets serve distinct diagnostic purposes. Table B1 makes their relationship explicit; these diagnostic subsets do not replace the full-benchmark pass@1 results.

Table B1: Temporal diagnostic subsets and their non-overlapping purposes.

# Questions	Purpose
321	Official LoCoMo raw-category-2 temporal set used for category accounting and the frozen-output error audit.
319	Category-2 questions with author-annotated gold-fact mappings for candidate coverage; two questions without reliable mappings are excluded.
82 + 237	Disjoint development and held-out partitions of the 319 mapped questions; only the 82-question split selects Log-TimeRerank and lightweight MemTree-selector weights.
249	Separate cross-benchmark LoCoMo/LongMemEval routing and fragmentation audit, rather than a further split of the 321-question set.
231	Supported mappings retained after independent review and author adjudication: 130 LoCoMo and 101 LongMemEval questions; 18 unsupported rows are excluded with reasons.

B.2 Fixed-Retrieval Dual-Time Control

To isolate context rendering from retrieval, both arms use the same top-30 fact IDs for all 321 official LoCoMo temporal questions. The answer backbone is Qwen3-30B and the paired answers are judged with the same public LoCoMo judge. Table B2 shows a net gain of ten correct answers; this targeted diagnostic does not replace the main-table protocol or establish that the complete LoCoMo gap is removed.

Table B2: Dual-time rendering with retrieval IDs held fixed.

Context arm	Correct / 321	Pass@1	Paired change
Fact text only	245	76.32%	–
Fact text + dual time	255	79.44%	+14 / −4

Exact paired McNemar $p = 0.0309$. Dual time preserves source conversation time, resolved event time, the original temporal expression, and its resolution base.

B.3 Log-TimeRerank Development Grid

Log-TimeRerank scores each fact using semantic similarity plus weighted lexical and temporal features. We freeze an 82-question development split and a disjoint 237-question held-out split by question ID. The predefined grid is lexical weight $\{0, 0.04, 0.08, 0.12\}$ and temporal weight $\{0, 0.5, 1.0, 1.5\}$. Table B3 reports development top-10 macro coverage. The selected pair $(0.12, 1.0)$ is the best point in this grid. Because 0.12 lies on the grid boundary, the search does not establish an optimum beyond the predefined range. Across the 16 settings, development coverage ranges from 76.67% to 82.60%.

Table B3: Log-TimeRerank development macro coverage (%).

Temporal \Lexical	0	0.04	0.08	0.12
0.0	77.89	77.48	79.92	81.79
0.5	77.89	79.11	80.33	82.20
1.0	76.67	77.89	79.11	**82.60**
1.5	76.67	77.48	78.70	80.57

Selection sensitivity. For transparency, the lightweight MemTree selector is also selected on the same development split, over 96 predefined combinations of tree-prior, novelty, and redundancy weights. Its top-10 development coverage ranges from 79.07% to 82.40%, and the selected setting is $(0.30, 0.00, 0.00)$. Both selectors are tuned on the same development split, selected once, and frozen across all held-out budgets. The narrower observed MemTree range indicates lower sensitivity on these grids, while the held-out results establish that its advantage emerges when the pre-answer evidence budget is sufficient rather than at every k.

Table B4: Held-out macro gold-fact coverage across final evidence budgets (%).

Representation	5	10	20	30	50	100
Flat Log	71.8	79.1	84.7	86.4	89.0	94.6
Log-Time	71.8	80.3	84.7	87.3	91.0	95.1
Validity Interval	71.2	77.1	80.0	82.8	86.2	90.4
Scratchpad+Background	68.4	77.6	82.3	84.2	87.5	90.9
Triplet Graph	70.3	78.8	83.1	85.5	88.0	93.9
MemTree	71.0	79.6	84.0	87.4	92.0	96.4

B.4 Routing, Browse, and Fragmentation Diagnostics

Planner and Embed use different native candidate pools and the same lightweight semantic/time reranker; Planner adds one LLM browse call. The scene sweep rebuilds routes from frozen facts and embeddings, so its Jaccard score measures assignment stability.

Table B5: Compact routing and retrieval diagnostics.

Diagnostic	Result	Boundary
Active entity routing	124/127 precision PASS; 3/127 exceptions; exact-all 9/127	Entity trees are a selective precision overlay; other scopes remain available
Planner vs. Embed	+0.49 to +2.37 pp macro coverage over $k = 5 \ldots 100$	Quality–cost comparison with one extra LLM browse call
Scene threshold	4B mean/p05 $\geq$ 0.9997/0.9996; 30B $\geq$ 0.9650/0.8829	Perturbs $\theta = 0.84, 0.92$ around default 0.88
Specific-tree fragmentation	4/130 LoCoMo; 37/101 LongMemEval	Evidence may remain reachable through union recall and fallback

The fragmentation audit retains 231/249 supported mappings after independent review and author adjudication. Under embedding browse, fragmented versus unfragmented questions score 2/4 (50.00%) versus 96/126 (76.19%) on LoCoMo and 25/37 (67.57%) versus 50/64 (78.13%) on LongMemEval. The released audit records separate model-assisted labels, independent review, author decisions, exclusions, and validation summaries. Together, the audit supports high-precision active routes but not a single-route design:

Table C1: Chunk-sweep study for raw-fact extraction on the assembled-long-session benchmark.

Preset	Gold Coverage (%)	Facts/s	Token/fact
1-turn	100	7.40	868
2-turn	100	7.00	767
4-turn	100	1.90	811
8-turn	99	1.52	647
16-turn	94	1.37	682
32-turn	95	1.26	832
48-turn	88	0.62	1127
64-turn	72	0.46	1271
whole	55	0.52	2505

incomplete entity coverage and the LongMemEval fragmentation gap motivate redundant scope assignment, union recall, and multi-tree browse.

B.5 Extraction and Tree-Shape Sensitivity

The following configuration diagnostics support the defaults used by MemForest without using benchmark pass@1 to select the reported operating points.

C CHUNK-SIZE DIAGNOSTIC FOR RAW-FACT EXTRACTION

We study extraction chunk size in a separate diagnostic setting, since this operating-point choice is not the main contribution of MemForest and is therefore deferred from the main ablation section. The purpose of this experiment is to examine how raw-fact extraction degrades as chunk granularity increases.

To create a controlled stress setting, we manually assemble long sessions by concatenating multiple original conversations, then run the same raw-fact extraction pipeline under different chunk presets. This setup is therefore a diagnostic stress test rather than the benchmark's native dialogue layout, and is intended to expose how the extraction pipeline behaves as chunk granularity is varied.

We evaluate each setting using **Gold Coverage**, a diagnostic retention metric rather than an end-to-end QA metric. For each question, we identify its gold supporting turn range and the key answer-bearing span within that range, such as an entity, date, number, or short attribute phrase. A question is covered if at least one extracted fact produced from a chunk intersecting the gold range still preserves that key span after normalization.

Table C1 shows a clear constrained operating point. Whole-session extraction is both the least faithful and one of the least efficient settings, while chunk sizes beyond 8 turns show visible fidelity degradation. Very small chunks preserve answer-bearing information, but 2-turn extraction provides the best overall balance: it preserves 100% Gold Coverage, remains near the best throughput regime, and improves token efficiency over 1-turn extraction. We therefore use 2-turn extraction as the default write-path setting in the main system.

MemTree branching factor. Figure C1 separates the two constraints behind the default branching factor. Increasing k makes the tree shallower, but eventually asks one autoregressive summary call to preserve too many child facts. The selected moderate operating region balances dependency depth against summary retention; it is not tuned on the benchmark answer judge.

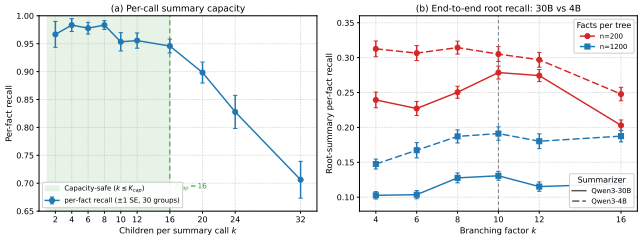

Figure C1: MemTree branching-factor diagnostics. (a) Per-fact retention in one summary call remains high through a moderate number of children and then declines. (b) End-to-end root-summary recall peaks at moderate branching factors across two tree sizes and two summarizers. Here k denotes tree branching factor, not final retrieval top-k; these controlled summary-retention measurements are not benchmark pass@1.

D SYSTEM COST, SCALING, FRESHNESS, AND MIGRATION

D.1 Matched Request-Level Token Probe

The same three-stream trace averages 435.7K input and 451.9K output tokens over 258.3 extraction calls, and 177.2K input and 77.6K output tokens over 320.0 dirty-refresh calls. In matched historical service logs, 46 stable windows with one running request, no queued request, and no prompt processing sustain a median 140.8 output tokens/s (p10–p90: 140.2–145.9). Request-level fits give effective prompt rates of 2.67K–21.85K tokens/s, depending on prefix reuse. Applying $T_{\text{serial}} = T_{\text{in}}/r_{\text{prefill}} + T_{\text{out}}/r_{\text{decode}} + n_{\text{req}}t_{\text{fixed}}$ defines a throughput-calibrated serial replay of the logged request stream. Across the observed prefill-rate range, replay takes 52.76–56.41 minutes for extraction and 9.23–10.52 minutes for dirty refresh. Figure 8 uses the respective midpoints, 54.58 and 9.88 minutes, versus measured parallel walls of 122.31 and 27.92 seconds (26.8× and 21.2× speedups). This replay holds the request stream fixed and sets the maximum in-flight request count to one; eager per-insert refresh would instead change the request sequence.

Under this self-hosted vLLM scheduler, prefix reuse reduces MemForest prefill relative to EverMemOS but not Mem0. Request length alone does not explain the result: MemoryOS has the largest p95 lengths, while Zep Local makes far more requests. MemForest's measured advantage comes from parallel extraction cells and same-level, cross-tree refresh waves. Caching reduces repeated prefill but not dynamic prefill or autoregressive decoding; the self-hosted deployment has no per-token API bill.

The DeepSeek billing probe measures a separate provider-billed cache metric. We verified that the ten MemForest extraction requests in the Qwen/vLLM and DeepSeek probes have the same ten prompt hashes and the same fixed 6,014-character system prefix. They are submitted as one burst (9.7 ms and 17.3 ms start-time spans, respectively). vLLM reports cache hits for nine of these ten

Table D1: Self-hosted vLLM APC request diagnostic over 20 matched LoCoMo messages. Token p95 uses the nearest-rank definition. Cached input is measured from `prompt_tokens_details.cached_tokens`: it is vLLM avoided prefill, not a provider-billed cache class or a semantic prompt partition.

Method	Req.	Input avg.	Input p95	Uncached avg.	Uncached p95	Output avg.	Output p95	Cache share
MemForest	19	1,196	2,482	498	2,482	267	670	58.38%
EverMemOS	29	1,098	1,594	897	1,594	187	380	18.29%
Mem0	13	1,306	1,741	515	1,520	155	459	60.59%
MemoryOS	40	482	2,837	366	1,941	167	1,474	24.07%
Zep Local	190	723	1,536	317	1,124	58	195	56.14%

Table D2: MemForest write-stage breakdown (179.74 seconds total).

Stage	Time	Share	Mean calls	Main work
Extraction	122.31 s	68.05%	258.3 LLM	LLM
Canonicalize/dedup	12.57 s	6.99%	31.3 LLM + 49 emb.	Hybrid
Routing	15.54 s	8.64%	5.7 emb.	Embedding + CPU
Bucketize/insert	0.03 s	0.02%	0	CPU
Dirty refresh	27.92 s	15.53%	320.0 LLM	LLM
Index update	1.13 s	0.63%	3.0 emb.	Embedding
Persistence	0.24 s	0.13%	0	I/O

Table D3: Mean provider-billed token use and direct cost over three probes.

Method	Cached in	Miss in	Output	Total	Cost (¢)	Eff.
MemForest	15.45k	7.51k	3.64k	26.60k	0.212	5.28×
EverMemOS	3.37k	24.57k	3.45k	31.40k	0.442	2.53×
Mem0	13.65k	4.44k	1.27k	19.37k	0.102	11.00×
MemoryOS	1.88k	8.84k	2.44k	13.15k	0.193	5.81×
Zep Local	47.66k	66.77k	6.06k	120.48k	1.118	1.00×

Cost efficiency is normalized to Zep Local. Costs use provider-returned billable token classes and the official non-thinking DeepSeek-V4-Flash price [6].

Table D4: Cold-start versus verified warm-cache DeepSeek billing. Hit share is provider-reported hit input divided by total input; cost is US cents per 20-message native probe. Warmups are excluded from both columns.

Method	Cold hit	Warm hit	Cold cost	Warm cost
MemForest	0.00%	67.28%	0.413	0.212
EverMemOS	1.83%	12.06%	0.479	0.442
Mem0	50.60%	75.47%	0.166	0.102
MemoryOS	7.27%	17.52%	0.225	0.193
Zep Local	38.60%	41.65%	1.138	1.118

requests (12,944 avoided-prefill tokens), whereas DeepSeek reports zero. This is consistent with backend semantics rather than a lost accounting field: DeepSeek requires a complete persisted prefix unit, its common-prefix example first identifies the prefix after two requests complete, and cache construction can take seconds [5]. Therefore Table D1 characterizes local serving work, while Table D3 alone determines the reported API bill; cache shares and rankings are not compared across the two backends.

Table D5: Embedding retrieval as frozen state grows.

Forests	Facts	Trees	Root rows	p50	p95
1	1,428	250	210	37.63 ms	41.79 ms
2	2,694	463	369	39.14 ms	43.58 ms
4	5,553	950	754	39.10 ms	44.13 ms
8	10,781	1,868	1,455	39.24 ms	42.23 ms

The cold-to-warm change is largest for MemForest because its long extraction instruction is shared by one concurrent first wave: vLLM can reuse scheduled blocks within that wave, while DeepSeek cannot bill a hit until the prefix unit has persisted. We use the verified warm-cache result for steady-state cost efficiency and report cold-start behavior separately.

For API billing, we probe three distinct LoCoMo conversations (`conv-26`, `conv-42`, and `conv-43`), truncate each to 20 messages, and rebuild every method from an empty store. Two additional conversations, `conv-47` and `conv-48`, warm each method under the same isolated provider `user_id` and are excluded from measured totals. We verify cache-eligible fixed templates; the second native warmup produces a nonzero hit for every method. All 15 measured cells complete without failed requests. Table D3 reports an unweighted mean because every probe has the same message count; per-conversation records are released with the probe. MemoryOS makes 34–38 requests and consumes 9.09k–18.93k total tokens because its LLM-generated session grouping changes how often the heat-triggered profile/knowledge update fires.

D.2 Frozen-Index Query Scaling

The sweep uses fixed top-10 embedding browse and excludes planner calls, answer generation, index loading, and memory construction. A separate synthetic structural sweep varies both facts per tree and number of trees. Across the two complete 500-question Qwen builds, the audit covers 250,118 trees; observed maximum height is four and no tree violates the conservative bounded-fan-out height check.

Table D6: Session-level entity/scene-tree write concentration.

Scope	Trees/sess.	Facts/tree mean/med/p95/max	Within	Across
All trees	7.60	5.39/3/19/53	67.1%	89.3%
Specific trees	6.65	5.04/2/21/53	63.7%	16.6%

"Specific" excludes the global `entity:user` fallback. Within is the share of session–tree buckets containing at least two facts; Across is the share of within-user session pairs sharing at least one tree.

D.3 Freshness Contract and Write Concentration

Section 4.5 states the online tree-level concurrency contract. Each asynchronous B+ tree has an independent reader–writer lock: insertion and refresh are writers, whereas range query, browse primitives, and state export are readers. Distinct trees and concurrent readers therefore proceed independently; only a writer and another operation on the same tree conflict. The online auto-update path refreshes dirty trees and root vectors before returning. Snapshot save exports each tree under its read lock, writes a temporary directory, and renames the completed snapshot into place. The released source snapshot contains the evaluated lock and scheduler implementation.

Session-level write concentration. We attribute canonical facts to their first source session and join them to the final entity and scene trees in the same three representative LongMemEval streams used by Figure 8. Table D6 reports all 147 sessions. A within-session collision is a session–tree bucket containing more than one fact. Across-session overlap compares session pairs only within the same independent user stream. The released audit directory contains the 1,117 session–tree rows, manifest, summary, and regeneration script.

Repeated same-session assignments do not trigger one LLM refresh per fact: each tree is marked dirty once and shared ancestors are summarized once per flush. Across sessions, 16.6% of pairs share a specific entity/scene tree; including the global `entity:user` fallback raises overlap to 89.3%, making it the primary lock hotspot. This fallback can be partitioned into independently built subforests and consolidated through the same migration merge evaluated in Section 5.6.

Provenance links extraction errors to affected leaves and derived artifacts for targeted deletion and refresh.

D.4 Migration-State Scaling

Sequential replay and migration merge produce comparable persistent states: facts differ by less than 1% and trees by at most about 8%. The procedures need not be bit-identical because extraction, routing, and summary refresh contain model or heuristic decisions, but the measured migration speedup does not come from collapsing or discarding memory state.

E DETAILED WRITE-PATH AND PARALLELISM ANALYSIS

This appendix details the released write-path dependencies summarized in Section 5.4. One complete memory instance contains N method-native input units and has a per-instance LLM worker budget P. LightMem consolidation uses $P = 5$, while MemForest uses

Table D7: Memory scale under sequential write and migration merge.

N	Seq Facts	Mig Facts	Seq Trees	Mig Trees
1	1,435	1,435	249	249
2	2,692	2,716	375	405
3	4,061	4,098	562	607
4	5,540	5,590	733	792
5	6,860	6,922	895	967
6	8,101	8,174	1,044	1,128
7	9,426	9,511	1,200	1,296
8	10,705	10,801	1,362	1,472

$P = 512$. A star denotes dependency-feasible parallel calls whose released per-instance ingestion path is serial. Because APIs batch dialogue differently, N denotes native units rather than equal-size text chunks; wall-clock latency is measured on complete question or conversation stores.

The comparison reports autoregressive-LLM dependency depth, not constant-time token work. Prompt/output length, CPU and database work, serving saturation, and persistence remain measurable costs. Extraction creates the method's first persistent memory representation; maintenance reconciles, summarizes, or indexes it. A stage is LLM-based when an autoregressive result lies on its commit path.

E.1 Method-Specific Dependency Boundaries

Mem0. One add extracts facts, retrieves candidates, and uses a second LLM to choose add/update/delete/no-op actions. Isolated extraction prompts are independent, but the evaluated adapter awaits the complete stateful add before issuing the next. Concurrent adds can reconcile against and mutate the same old record; decoupling extraction would require ordered commits, per-record locking, or optimistic validation. We therefore report the implemented $O(N)$ add chain while marking extraction as dependency-feasible parallel work.

MemoryOS.. Raw QA append is non-LLM, while queue eviction can trigger page continuity, meta-summary, profile, and knowledge updates. Profile and knowledge analysis already use two workers within a trigger, but both depend on selected mid-term pages and same-user queue/profile commits remain ordered. A hot profile can also make a triggered LLM input grow with history.

EverMemOS.. Streaming boundary detection observes unresolved conversation history, so later boundaries depend on earlier decisions and one conversation has $O(N)$ ordered extraction depth. Once MemCells are frozen, embedding and indexing distinct cells add no history-dependent LLM call and can be scheduled concurrently. Optional foresight/profile extractors are disabled in the evaluated configuration.

LightMem. Buffer accumulation and segmentation are ordered, but emitted segments could be extracted in bounded parallel waves; the released benchmark runner instead awaits each `add_memory`. Its maintenance workers summarize independent entries from one frozen candidate snapshot and serialize only non-LLM mutations

Table E1: Detailed write-path decomposition. Complexity denotes logical dependency depth. A star marks dependency-feasible parallel calls whose released per-instance ingestion path is serial.

System	Extraction path	LLM	Maintenance path	LLM
Mem0	History-independent extraction: $O(\lceil N/P \rceil)^*$; released add loop $O(N)$	Yes	Candidate search and reconciliation per add; $O(N)$ ordered commits	Yes
MemoryOS	Raw QA append; $O(N)$ ordered queue operations	No	At most N triggered page/meta-summary and profile updates, each with up to $O(N)$ profile input	Yes
EverMemOS	Boundary detection and MemCell formation; $O(N)$ ordered stream	Yes	Embedding/index persistence for at most N emitted cells; $O(N)$ work	No
LightMem	Post-segmentation extraction: $O(\lceil N/P \rceil)^*$; released add loop $O(N)$	Yes	Frozen-snapshot consolidation: $O(\lceil N/P \rceil)$ independent LLM waves	Yes
MemPalace	Deterministic chunk/drawer formation; $O(N)$ non-LLM work	No	Embedding and append/index insertion; $O(N)$ work, no semantic rewrite	No
Zep Local	Node/edge extraction per episode; $O(N)$ ordered episode chain	Yes	$O(N)$ ordered LLM chain; backend search $\sum_i g(G_i)$, index- and graph-dependent	Yes
MemForest	Implemented gather plus semaphore; $O(\lceil N/P \rceil)$ request waves	Yes	$O(\lceil N/P \rceil)$ bounded-worker waves plus $O(\log N)$ dependent levels	Yes

under a write lock. Thus the evaluated extraction path is serial, whereas its snapshot-based LLM maintenance already uses $P = 5$ workers.

MemPalace. Chunk and drawer creation, embedding, and index insertion use no autoregressive LLM and perform $O(N)$ work. Preparation and embedding can be parallelized subject to ordered identifier/timestamp commit. Zero LLM depth does not imply zero CPU or database cost, and the method does not perform LLM temporal reconciliation on write.

Zep Local / Graphiti. Native `add_episode` extracts and resolves nodes/edges, invalidates superseded facts, and commits graph objects. Episodes in one namespace are awaited sequentially because later resolution observes the updated graph; within-episode operations and independent namespaces can still run concurrently. We denote indexed candidate-search and graph-query work before episode i by $g(G_i)$ because it depends on graph size, indexes, and query plans. The ordered LLM chain buys versioned facts, provenance, and topology and is not an asymptotic claim about every Graphiti deployment.

MemForest. Extraction cells do not read accumulated memory and take $O(\lceil N/P \rceil)$ request waves. For M_s new facts in scope s, structural CPU work is $\sum_s O(M_s \log N_s)$. Dirty-summary regeneration performs $O(|D|)$ total LLM work but allows same-level and cross-tree nodes to run together, giving

$$O\left(\lceil N/P \rceil + \max_{s \in S} \log N_s\right)$$

request-wave depth under balanced bounded-fanout trees. Routing, structural insertion, index update, and persistence are non-LLM. With fixed P, total construction remains $O(N)$; the contribution is bounded parallel waves and localized dependency, not sublinear total work.

The table analyzes schedules and state dependencies in the released paths evaluated here. Redesigned parallel schedulers are outside its scope.

F LONGMEMEVAL DIAGNOSTIC SUBSET FOR ABLATION

For the retrieval ablations in Section 6.2, we construct a balanced 60-question diagnostic subset by sampling 10 questions from each LongMemEval question type. Table F1 lists the full question IDs.

Table F1: Question IDs in the 60-question LongMemEval diagnostic subset (10 per type).

Question Type	Question IDs
Knowledge Update	`01493427`, `031748ae`, `0977f2af`, `18bc8abd`, `1cea1afa`, `4d6b87c8`, `69fee5aa`, `852ce960`, `a1eacc2a`, `f9e8c073`
Multi-Session	`1a8a66a6`, `60bf93ed_abs`, `73d42213`, `81507db6`, `88432d0a_abs`, `aae3761f`, `d682f1a2`, `e5ba910e_abs`, `gpt4_d84a3211`, `gpt4_f2262a51`
Single-Session (Assistant)	`0e5e2d1a`, `1568498a`, `16c90bf4`, `561fabcd`, `6222b6eb`, `7161e7e2`, `71a3fd6b`, `7a8d0b71`, `8cf51dda`, `c7cf7dfd`
Single-Session (Preference)	`06f04340`, `0edc2aef`, `195a1a1b`, `1a1907b4`, `1d4e3b97`, `35a27287`, `6b7dfb22`, `75832dbd`, `a89d7624`, `caf03d32`
Single-Session (User)	`001be529`, `311778f1`, `545bd2b5`, `5d3d2817`, `60d45044`, `94f70d80`, `af8d2e46`, `c5e8278d`, `caf9ead2`, `faba32e5`
Temporal Reasoning	`4dfccbf7`, `982b5123`, `e4e14d04`, `gpt4_18c2b244`, `gpt4_1e4a8aec`, `gpt4_2487a7cb`, `gpt4_68e94287`, `gpt4_7a0daae1`, `gpt4_e072b769`, `gpt4_f420262d`